ForesightBench: Benchmarking Research Foresight in Large Language Models

Ruhaan Chopra

Independent Researcher
ruhaanchopra2005@gmail.com

Abstract

Mainstream autoresearch loops largely search by brute force, which works only while the search space is small. Beyond that threshold, an agent needs research taste: a sense of which experiments will pay off before running them. To assess frontier models on their ability to forecast empirical outcomes of given research setups, we introduce ForesightBench, built by an agentic pipeline that turns recent arXiv preprints into contamination-controlled benchmark samples, and evaluate fourteen frontier models on 42 samples from papers submitted after every published training cutoff. We reveal that while most models get the direction of the prediction right, they are consistently conservative in their predictions. Error is also highest on safety and interpretability papers (0.44) and agent papers (0.42), against 0.17 on vision. We then identify three failure modes of reasoning, and discover that faulty predictions often happen when models fail to take cognizance of the entire provided experimental setup and imprudently rely on their priors. Using this as our guiding principle, we introduce a lightweight inference-time intervention which decomposes the setup into threads and sub-threads. We test this harness built on top of pi against vanilla, out-of-the-box pi, tasking them both with creating a long-horizon agent. Across two seeds, the agent built by our custom harness outperforms the one built by vanilla pi on DABstep by 8 percentage points on average.

Introduction

Modern frontier LLMs are multi-trillion-parameter models (Fedus, Zoph, and Shazeer 2022; Kimi Team 2026) that hold vast swaths of knowledge absorbed in their parametric memory and maintain a deep record of human endeavour (Allen-Zhu and Li 2024; Calderon et al. 2026). Their capabilities are rapidly advancing in domains like coding (Kwa et al. 2025; Desai et al. 2026), and recent work has attempted to leverage these capabilities to advance scientific research (Lu et al. 2024; Novikov et al. 2025).

Specifically in the AI research domain, autoresearch is a growing paradigm that involves having agents come up with breakthroughs which outperform the baseline (Karpathy 2026; Kong et al. 2026). It started as a primitive setup, where an agent with access to compute iterated over a `train.py` file until a required metric (e.g., training loss below a threshold) was fulfilled, building on a line of ML-engineering agents and benchmarks (Wijk et al. 2024; Chan et al. 2025). Over the past year, autoresearch pipelines and detailed recursive self-improvement (RSI) loops have become commonplace (Weng et al. 2025; Ding et al. 2026).

But the core iteration still seems to be carried out by brute force, which becomes increasingly inefficient as the search space grows, at which point the process must rely on the agent's research taste to determine what to explore and what to ignore. Research taste is a nebulous concept and is notoriously hard to quantify. Therefore, we need the closest quantifiable proxy: a verifiable metric that is downstream of tasteful experimentation. We determine that to explore the search space fuzzily, an agentic system must have the capability to identify the specific research directions out of all available ones which will end up being the most fruitful. In other words, the underlying model must possess the reasoning ability required to predict the empirical outcomes of the research setup provided. This need for inculcating capabilities to forecast the results of future directions before committing has also been documented in previous work (Foster et al. 2026).

To benchmark the capability of large language models to predict empirical results of research setups, we introduce **ForesightBench**, built by an agentic pipeline that scrapes recent arXiv preprints and turns each controlled within-paper comparison into a sample. Each sample consists of the paper's introduction and methodology with all results removed, and the task is to predict the value each reported metric takes after the technique is applied.

We evaluate fourteen frontier models on a 42-sample held-out set (Table 4), and reveal that on our scoring metric, the models often get the direction of change right (Table 5), but are consistently conservative and undershoot in their predictions (Table 6). We also notice that on our scoring metric, error is consistently high for safety and mechanistic interpretability preprints and lowest on vision research (Table 9). We show that faulty predictions often occur when a model fails to account for all moving parts of a given setup. Using this as our guiding principle, we design a lightweight agentic harness structured as a recursive fan-out tree, which decomposes the given brief into threads and sub-threads.

This harness is built on top of pi (Zechner 2026), and to demonstrate the effect of this inference-level intervention, we task both it and vanilla pi with building a long-horizon agent. The resulting agents are evaluated on the DABstep benchmark (Egg et al. 2025); averaged over two seeds, the agent built by our custom harness passes 46% of the held-out tasks against 38% for the agent built by vanilla pi, a gain of 8 percentage points (Table 10).

Benchmark Design and Statistics

Recent work has put LLMs in a forecasting position over research outcomes, but almost always through discrete choices. The model is asked which of two ideas will score higher on a benchmark (Wen et al. 2025; Mule, Garikaparthi, and Patwardhan 2026), which of two abstracts reports the real result (Luo et al. 2025), whether an advance will be realised or which of several candidate pathways or directions will pan out (Wu et al. 2026; Tian et al. 2026), or to pick from three or four options, with numeric items scored only as inside or outside an interval (Sehwag et al. 2026). These framings are brittle in a few ways: a choice between options implies that at least one is right, no baseline is given to anchor the answer, and a correct pick might not say anything about how large the effect was. Continuous targets in prior work take a different form from ours: PRECOG forecasts an absolute benchmark score from a masked description of the method (Park et al. 2025), and Zheng et al. compare agent-written code pairwise before execution (Zheng et al. 2026).

We instead treat the LLM as an assistant to a researcher who has an exact methodology and experimental setup in mind, has already run the baseline on named benchmarks, and is mulling the idea over with the assistant rather than asking it to pick a winner. The prompt gives the full setup, the benchmarks, and the baseline numbers, and asks for the value each metric will take under the proposed method.

Because the answer is a number against a known baseline, every prediction implicitly carries a direction and a magnitude axis, which exposes behaviours a binary choice hides, such as conservatism toward the baseline or systematic over- or undershooting.

ForesightBench samples are built with our retrieval pipeline[1] (Appendix A; Figure 1 walks through one sample), which turns each controlled within-paper comparison in a recent arXiv preprint into a prompt with the results removed and a graded target. The papers come from a fixed arXiv submission window, which is the contamination control: for this corpus every paper was submitted between 8 and 24 August 2026, after the latest published training cutoff (June 2026) of every model whose vendor publishes one, and web search is disabled at inference. Three models (Grok 4.6, DeepSeek V4 Pro 0813, GLM-5.3) were released between 12 and 14 August without a published cutoff; we assume their training data realistically excludes papers submitted one to six days before release.

[1]https://github.com/ruhzi/autoresearchrl

Every sample produced by the agentic pipeline was then audited by hand before entering the corpus, and we recommend the same manual audit to anyone using the pipeline to build their own foresight benchmark. The system prompts of every LLM stage, including the retrieval agent, are reproduced verbatim in Appendix B. A finished sample begins "I've been spending cycles on isolating how much" and ends with a question asking for the value of each (benchmark, metric) pair after switching from the baseline arm to the method arm; one complete sample with its target is reproduced verbatim in Appendix C.

Corpus and Evaluation Set

A single preprint can yield several distinct samples, one per controlled comparison it reports: each ablated component, each technique it introduces, and each setting it is tested under is treated as its own sample, because each different setting introduces a unique experimental context. The ForesightBench corpus consists of 316 samples from 42 preprints (Table 1); the corpus and the evaluation set are released together on GitHub[2] and Hugging Face.[3] Each sample asks for one or more graded slots, one per (benchmark, metric) pair, 2,052 in all.

The median preprint contributes 4.5 samples; seven preprints contribute a single sample, and the most, 29, come from one preprint's ablation grid (arXiv 2608.09228; Ichihara et al. 2026). The evaluation set draws exactly one sample per preprint, chosen by a random seed, giving 42 samples with 268 slots. The sign of the true change is close to balanced: the method value is higher than the baseline on 47% of evaluation slots, lower on 44%, and equal on 9% (Table 1), so a constant guess on the raw direction of change is right about half the time. Whether a higher value is better depends on the metric and is not tagged in the corpus, so this is a statement about the sign of the change, not about whether the method helped. We tag every preprint by hand with one of six research areas from its title and abstract (Table 2; definitions and per-preprint assignments, with a citation for every source preprint, in Appendix D).

Evaluation

Models and Generation Settings

Fourteen frontier models were run on the ForesightBench evaluation set through OpenRouter with web search disabled, reasoning on at the provider's high effort setting, temperature 0.4, an output budget of 32,768 completion tokens that no completion exhausted (the longest was 30,158 tokens, DeepSeek V4 Pro), and one completion per sample (Table 3). The same 42 prompts, with the same system turn (Appendix C), were also run on Qwen3.5 4B, served locally with vLLM on one L4 GPU. It was evaluated under its model card's recommended thinking-mode preset for general tasks: temperature 1.0, top-p 0.95, top-k 20, presence penalty 1.5, thinking enabled, the same 32,768-token budget (its longest completion was 7,175 tokens), one completion per sample.

[2]https://github.com/ruhzi/benchmark

[3]https://huggingface.co/datasets/ruhzi/foresightbench

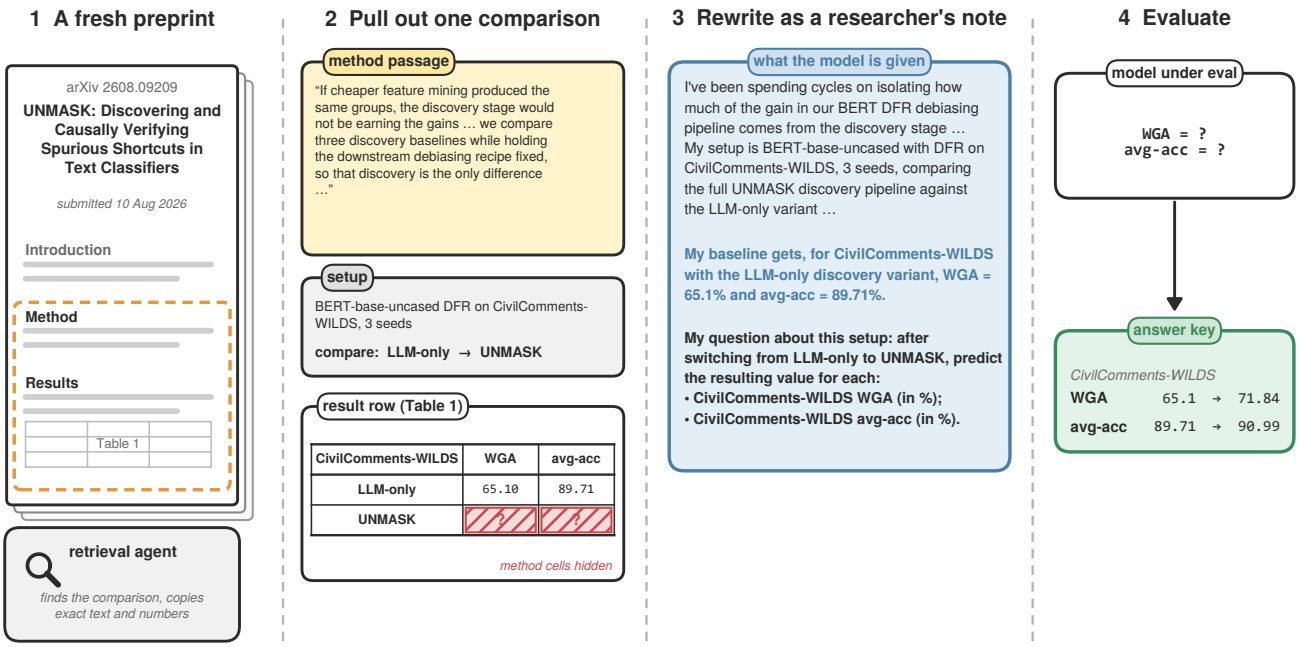


Figure 1: How one ForesightBench sample is built, shown on sample `2608.09209-s051` (Ravuru and Srivastava 2026). A retrieval agent pulls one controlled comparison out of a fresh preprint: the method passage, the experimental setup, and the result row, with the method cells hidden. The copied text and the baseline numbers are rewritten into a first-person researcher's note that ends with the question, and the hidden method cells become the answer key.

Table 1: Corpus and evaluation-set statistics. Samples per preprint are 1 in the evaluation set by construction.

	Corpus	Eval set
Preprints	42	42
Samples	316	42
Graded slots	2,052	268
Samples per preprint: mean (sd)	7.5 (7.4)	1
median	4.5	1
min / max	1 / 29	1 / 1
IQR outliers (preprints)	3	0
share of samples in top 3 preprints	24.7%	7.1%
Slots per sample: mean (sd)	6.5 (7.2)	6.4 (6.2)
median	4	4
min / max	1 / 56	1 / 32
Slots where the method value is higher	51.6%	47.4%
lower	44.6%	44.0%
equal to the baseline	3.8%	8.6%

Table 2: Thematic coverage. Area is a per-preprint tag applied to all of its samples.

Area	Preprints	Corpus	Eval set
Vision & multimodal	9	101	9
Safety, attacks, interpretability	9	61	9
Agents, memory & context	8	27	8
Applied & scientific ML	8	65	8
LLM training & architecture	5	56	5
LLM evaluation studies	3	6	3

Scoring

The score estimates how far, on average, the model's prediction is from the reported value. Each graded slot has a true method value v and a prediction $\hat{v}$, and its error is the relative distance between them, capped at one:

$$e = \min\left(\frac{|\hat{v} - v|}{\max(|v|, \epsilon)},\ 1\right), \qquad \epsilon = 10^{-6}, \qquad (1)$$

and a slot the model leaves without a number scores 1. The number reads directly: 0 is exact, 0.3 is 30% off the reported value, and 1 is off by the value's own size or worse. A sample's error is the mean of e over its slots and a model's error is the mean over its samples, so every sample carries equal weight however many slots it asks for. Lower is better.

Empty slots are handled by who caused them. Slots lost on the serving path (provider content filters, transport errors, empty output with leaked control tokens) are excluded for that (model, sample) pair only; slots the model itself leaves without a number are charged the cap of 1. Provider content filters account for every exclusion on the Claude models and one serving fault for the single exclusion on DeepSeek V4 Pro, so Claude Opus 5.5 and DeepSeek V4 Pro are scored on 41 samples, Claude Fable 5.1 on 39, Claude Fable 5 on 38, and every other model, the local Qwen3.5 4B included, on all

Table 3: Generation settings per model.

Model	Via	Temp.	Reasoning	Budget	n
GPT-6 Astra	OpenRouter	0.4	high effort	32,768	42
Claude Opus 5.5	OpenRouter	0.4	high effort	32,768	41
GPT-5.6 Sol	OpenRouter	0.4	high effort	32,768	42
DeepSeek V4 Pro	OpenRouter	0.4	high effort	32,768	41
Grok 4.6	OpenRouter	0.4	high effort	32,768	42
GPT-6 Sol	OpenRouter	0.4	high effort	32,768	42
Inkling	OpenRouter	0.4	high effort	32,768	42
Kimi K3	OpenRouter	0.4	high effort	32,768	42
Claude Fable 5.1	OpenRouter	0.4	high effort	32,768	39
Beam 1.0	OpenRouter	0.4	high effort	32,768	42
DeepSeek V4 Flash	OpenRouter	0.4	high effort	32,768	42
GLM-5.3	OpenRouter	0.4	high effort	32,768	42
Claude Fable 5	OpenRouter	0.4	high effort	32,768	38
GPT-6 Luna	OpenRouter	0.4	high effort	32,768	42
Qwen3.5 4B (local)	vLLM (local)	1.0	thinking	32,768	42

42 (the n column of Table 3). Mean completion tokens per scored sample are reported beside every score; the count is the provider's billed completion total, which includes hidden reasoning tokens for every model.

As a reference, Table 4 closes with a *copy-the-baseline* row: the predictor that answers the baseline value on every slot, which amounts to assuming the intervention changed nothing.

Because every slot also carries its baseline value b, a prediction decomposes into a direction and a size of move off the baseline, and we report both beside the error. The *direction* of a slot is right when $\hat{v} - b$ and $v - b$ have the same sign; it is evaluated on slots where the reported value moved ($v \neq b$), and a prediction that stays on the baseline counts as neither right nor wrong. The *move ratio* of a slot is the size of the predicted move relative to the reported one, with sign ignored:

$$r = \frac{|\hat{v} - b|}{|v - b|}, \tag{2}$$

so $r = 1$ is a move of exactly the reported size, $r < 1$ an undershoot and $r > 1$ an overshoot, whichever way the prediction went. It is computed on slots where both the reported value and the prediction moved. We report the share of such slots with $r < 1$ and the median of r, once over all such slots and once over the subset whose direction is also right.

Results

The results are revealing, and a striking behavioural trait is conservatism. Every one of the fifteen models undershoots more often than it overshoots (Table 6): on the slots where both the reported value and the prediction moved off the baseline, the predicted move is smaller than the reported one on 52% to 67% of slots, and the median predicted move covers only half to nine tenths of the real one. The models tend to be largely directionally correct (65% to 82% of slots, Table 5) but conservative in their final empirical predictions.

Mean per-sample error on the 42-sample evaluation set lies between 0.300 and 0.347 for all fifteen models (Table 4; medians by area in Table 14, Appendix E). Copying the baseline, which is to say assuming the intervention did nothing, has a mean error of 0.374.

GPT-6 Astra is the clearest outperformer. It has the lowest mean error (0.300; Table 4), it is the boldest model in the field (undershoot on 52% of slots, a median move of 0.91 of the reported one; Table 6), and it is second only to Kimi K3 in directional accuracy (80% against 82%; Table 5). Yet even Astra undershoots more often than not. At the other end, GPT-6 Luna and the local Qwen3.5 4B are the most timid: both undershoot on 67% of slots and their median move is half the reported one (Table 6), and they sit at the bottom of the error table too (Table 4).

The number of reasoning tokens spent seems to have no notable effect on the result. Completion tokens per sample range from 1,038 for Claude Fable 5 to 11,011 for DeepSeek V4 Pro (Table 4; generation settings in Table 3), more than a tenfold spread in reasoning, and the extra reasoning yields no significant gains. The best model in the field, Astra, is also one of the most frugal at 1,536 tokens. Thinking longer does not make a model less conservative.

Table 4: Error by model.

Model	Mean	Median	Tok./sample
GPT-6 Astra	**0.300**	**0.169**	1,536
Claude Opus 5.5	0.307	0.219	1,160
GPT-5.6 Sol	0.316	0.179	2,756
DeepSeek V4 Pro	0.319	0.171	11,011
Grok 4.6	0.320	0.205	4,674
GPT-6 Sol	0.322	0.199	1,849
Inkling	0.327	0.208	4,633
Kimi K3	0.328	0.176	6,255
Claude Fable 5.1	0.328	0.193	1,254
Beam 1.0	0.330	0.214	4,426
DeepSeek V4 Flash	0.332	0.185	8,711
GLM-5.3	0.333	0.226	2,606
Claude Fable 5	0.336	0.248	1,038
GPT-6 Luna	0.340	0.194	2,010
Qwen3.5 4B (local)	0.347	0.215	2,348
Copy baseline	0.374	0.264	–

Kimi K3 predicts the direction of the intervention correctly most often, on 82% of slots, GPT-6 Astra on 80%, the rest of the frontier on 72% to 79%, and the 4B least often, on 65% (Table 5).

Notably, pooled over all models, error is highest on safety and interpretability papers (0.44) and agent papers (0.42), and lowest on vision (0.17) and LLM evaluation studies (0.17) (Table 7). The split holds for every model individually (Table 9): each model's error on vision and LLM evaluation papers is below its error on safety and agent papers, although

Table 5: Direction of predicted change.

Model	Direction right
Kimi K3	**82%**
GPT-6 Astra	80%
GPT-6 Sol	79%
Inkling	78%
DeepSeek V4 Flash	78%
GPT-5.6 Sol	77%
DeepSeek V4 Pro	77%
Grok 4.6	77%
Claude Opus 5.5	75%
GLM-5.3	75%
Claude Fable 5.1	74%
Claude Fable 5	74%
Beam 1.0	73%
GPT-6 Luna	72%
Qwen3.5 4B (local)	65%

Table 6: Size of the predicted move off the baseline relative to the reported move, sign ignored (move ratio r, defined under Scoring). Undershoot: share of slots with $r < 1$. Median ratio: median of r over slots where both the reported value and the prediction moved; last column restricts to slots where the direction is also right.

Model	Undershoot	Median ratio	Median ratio, dir. right
GPT-6 Astra	**52%**	**0.91**	**0.92**
DeepSeek V4 Pro	56%	0.78	0.84
Claude Opus 5.5	57%	0.79	0.82
Grok 4.6	57%	0.79	0.78
Claude Fable 5.1	57%	0.78	0.81
Claude Fable 5	57%	0.78	0.75
GPT-6 Sol	58%	0.79	0.77
DeepSeek V4 Flash	58%	0.83	0.81
GPT-5.6 Sol	62%	0.74	0.75
Kimi K3	62%	0.65	0.64
Beam 1.0	64%	0.76	0.80
GLM-5.3	65%	0.63	0.57
Inkling	66%	0.61	0.58
GPT-6 Luna	67%	0.50	0.46
Qwen3.5 4B (local)	67%	0.49	0.43

Table 7: Error and slots per area.

Research area	Samples	Slots/sample	Mean error
Vision	9	5.4	0.17
Safety & interpretability	9	4.2	0.44
Agents	8	6.5	0.42
Applied ML	8	8.4	0.38
LLM training & architecture	5	9.4	0.27
LLM evaluation studies	3	5.0	0.17
All	42	6.4	0.33

the order within each pair varies from model to model (area definitions in Appendix D).

After conducting a qualitative reading of every scored trace, we identified three failure modes of reasoning. Not every provider returns the chain of thought itself: the Anthropic, OpenAI and xAI models return a provider-written summary of the reasoning (the stored trace runs 0.4 to 1.3 characters per billed reasoning token, against roughly four for the models that return the full trace), so for those models the reading is of the summary and the visible answer.

Mode 1, *unfaithful chain of thought*: the committed conclusion is not a function of the reasoning that precedes it. On sample `2608.17941-s010` (Liu et al. 2026d), Kimi K3 works through the windowed estimator, writes "that argues for degradation, particularly in correlation", and then commits r above the baseline for both models; the true r fell from 0.836 to 0.712. This mode is well documented in recent work on chain-of-thought faithfulness (Chen et al. 2025; Jia, Benton, and Easley 2026; Gema et al. 2026).

Mode 2, *lapse in cognizance of the setup*: the trace fails to take the entire stated setup into account, often relying on priors that do not apply in that context, and the reasoning derails from there. On sample `2608.09577-s007` (Sui et al. 2026), where the prompt states that every trigger condition must carry a gate word, Kimi K3 prices the false-positive rate with "Given typical ablation tables: ... FPR ∼4–8%" and commits 6.0; the stated constraint drives it to 0.0.

Mode 3, *proposal bias*: beliefs about what the authors or the asker want influence the conclusion, in either direction. On sample `2608.10989-s023` (Cao et al. 2026a), Inkling considers that the attention-mean criterion may be no better than random, dismisses it with "why would they ask then?", and commits a smaller accuracy drop than random on all three tasks; the true drop was larger on two of them. This mode is well documented in recent work on sycophancy (Sharma et al. 2024; Feng et al. 2026; Nogueira et al. 2026).

Traces carrying mode 2 average an error of 0.56 against 0.27 for traces without it, a gap that holds for every model (Table 8; all three modes in Table 15, Appendix E).

Table 8: Mode 2 by model.

Model	Traces	Mode 2	With	Without
Claude Opus 5.5	41	**3**	0.60	0.28
GPT-6 Sol	42	**3**	0.68	0.29
GPT-6 Astra	42	4	0.61	0.27
Claude Fable 5	38	4	0.52	0.31
DeepSeek V4 Pro	41	6	0.49	0.29
Claude Fable 5.1	39	6	0.61	0.28
GPT-6 Luna	42	6	0.55	0.30
GPT-5.6 Sol	42	7	0.55	0.27
Beam 1.0	42	7	0.58	0.28
Grok 4.6	42	9	0.58	0.25
Kimi K3	42	9	0.61	0.25
GLM-5.3	42	9	0.65	0.25
Inkling	42	10	0.70	0.21
DeepSeek V4 Flash	42	10	0.53	0.27
Qwen3.5 4B (local)	42	27	0.45	0.17
All	621	120	0.56	0.27

19 of the 42 samples don't result in a single directionally

wrong judgment from any frontier model, while TRACE-Memory (Wang et al. 2026c) (83% of frontier calls wrong), attention-mean pruning (Cao et al. 2026a) (67%), MarkNull-A (Cao et al. 2026b), StagedWorkspace (Hua et al. 2026) and OPSD padding (Ichihara et al. 2026) (40% to 47%) result in high directional error. Two causes recur. Models assume the intervention/proposed method helps, so when it hurts (TRACE-Memory, attention-mean pruning, OPSD padding) nearly all of them predict a gain, a case of proposal bias (Mode 3). They also assume cheaper means worse, a misplaced prior (Mode 2) that fails in ACM+TA merging (Le et al. 2026) (cheaper and better) and Staged-Workspace (costlier and better, predicted cheaper and worse).

A full qualitative walk-through of one sample on which every model failed, with the prompt cues each trace acted on and every model's prediction, is given in Appendix G.

There is no correlation between the number of slots a sample asks for and the error on that sample, which suggests that the error is not a function of the number of predictions the model must make (Table 7; Figure 2).

Autoresearch-Harness

Dissecting the reasoning traces of these models and understanding where they fail at forecasting research outcomes lead us to theorise about the underlying principles we can derive from them. We derive the following driving principle, which follows from our analysis of failure modes and from our initial hypothesis that focusing on pre-experimentation forecasting would yield better autoresearch than current loops: the autoresearch loop must take cognizance of the entire setup before beginning implementation.

There are two ways to test this; the obvious one is a post-training environment designed around this principle. Budgetary constraints rule this out, so we instead test whether an inference-level intervention can have a sizeable effect. To that end, we design a lightweight agentic harness built on top of pi, and task it with a small autoresearch brief.

The harnesses (or autoresearchers) are tasked with creating a long-horizon agent, with a provided brief on the sort of tasks that agent must carry out (reproduced in full in Appendix F.1). We then evaluate the agents built by these autoresearchers on a set of held-out tasks, and compare the performance of the agents built by the harnesses.

The custom harness

The core tenet of the custom harness is that the researcher takes cognizance of the entire setup before it implements anything. The harness is vanilla pi (version 1.1.0) with one extension that translates this tenet into a fixed sequence of passes, run once before the researcher's first turn (Figure 3). Every prompt the extension adds is reproduced verbatim in Appendix F.

A root pass reads the task (the brief) and deduces its critical *threads*: the parts of the setup that the build depends on. Each thread must quote the brief's own words and state why it is crucial (system prompt and user messages in Appendix F.2). One further pass per thread then breaks it into sub-threads in the same way. The depth of this recursive fan-out tree is configurable; we chose a depth of two (threads and sub-threads) and a cap of six threads per pass.

The root conversation is then given one more turn in which it decides the chronology: the order in which the threads are to be built, by dependency (Appendix F.3). Implementation follows that order (the implementation turn and its stop steer are in Appendix F.4). Each first-level thread's conversation is continued into an implementation turn that builds its sub-threads in a git worktree of the workspace, one thread at a time, and every implementation turn receives the reports of the turns before it and may change earlier work if it names the change and the reason. The combined changes are held as a diff outside the workspace.

Meanwhile, the researcher itself starts from the plain task with one instruction: look around, change nothing, and reason out loud about what it would build, how it would test it and what could go wrong; edits are refused during that turn, and the pending output of the passes is unreachable from the workspace (Appendix F.5). When the researcher replies without tool calls, the diff is applied and the next turn makes it the final reviewer and integrator: it is told what was built and in what order, asked to hold the build to its own plan against both the thread tree and the code as built, and to change what conflicts (the hand-over message is in Appendix F.6). From there the run proceeds as an ordinary pi session until the budget is spent.

Setup

Both arms run inside the same sandboxed arena: the researcher container sees only its workspace and two model endpoints, and a gateway pins every call to a fixed model and provider. The outer (researcher) model is GLM 5.3 Flash (OpenRouter slug `z-ai/glm-5.3-flash`) in both arms. The inner model, which the built agent calls and which the researcher may spend on practice runs, is GPT-6 Luna (`openai/gpt-6-luna`); the inner budget is $4.00 of calls per run, and the run ends when it is spent or the researcher declares itself done. The brief describes the genre of task only (long-horizon quantitative questions over a workspace of files and their documentation) and never a benchmark item.

The agents are evaluated on DABstep, a benchmark of multi-step data-analysis questions over a shared set of files and documentation, graded against the reference answer with a numeric tolerance. We draw a held-out set of 75 tasks from its *hard* level, disjoint from the 50 tasks used to measure the seed agent while writing the brief. During evaluation each agent runs one fresh process per task on GPT-6 Luna with no network and no memory across tasks.

Each autoresearcher is run twice (seeds 1 and 2). A seed is a complete run of the outer autoresearcher, so each arm yields two independently built agents, and every agent is scored on all 75 tasks.

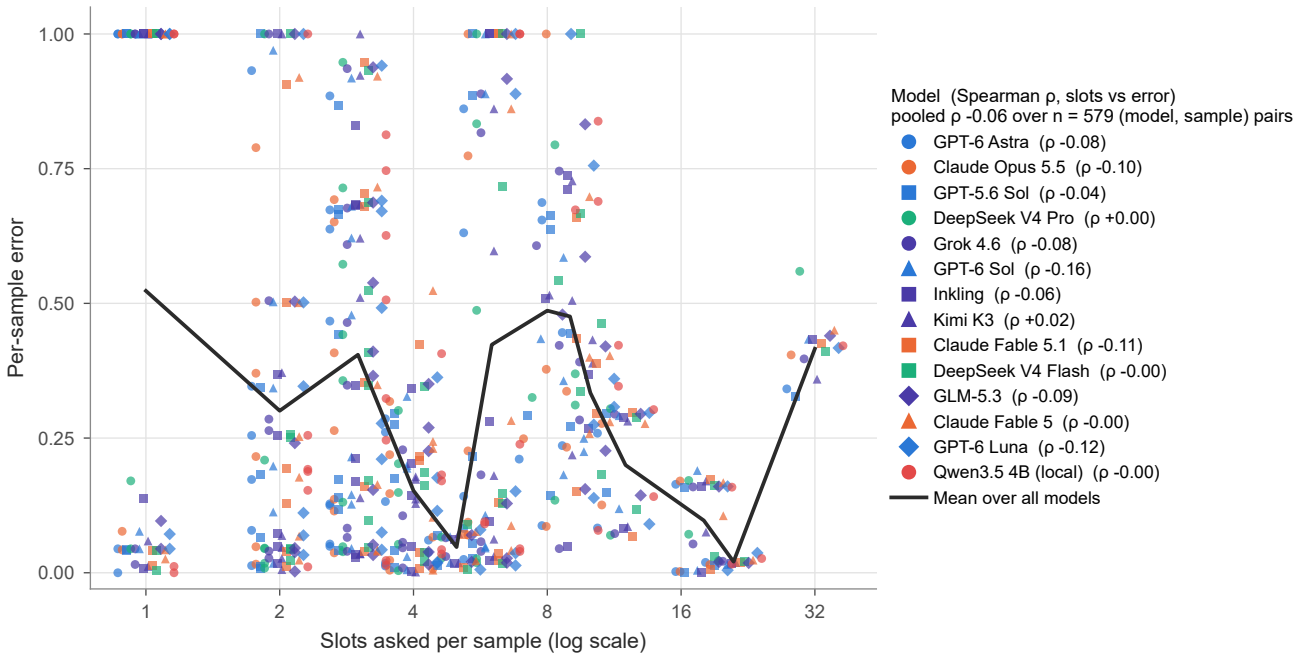


Figure 2: Per-sample error against the number of slots the sample asks for, every model in one panel. One point per (model, sample) pair; hue is the model family and marker the model, with a small horizontal offset per model so points at the same slot count do not overlap. The black line is the mean error over all models at each slot count. Spearman ρ between slots and error is given per model in the legend.

Table 9: Error by research area.

Model Samples	Vision 9	Safety 9	Agents 8	Applied 8	LLM-train 5	LLM-eval 3	All 42
GPT-6 Astra	0.15	0.40	0.37	0.35	0.28	0.15	**0.30**
Claude Opus 5.5	0.19	**0.36**	0.44	0.37	0.23	0.14	0.31
GPT-5.6 Sol	0.13	0.44	0.41	0.38	0.25	0.17	0.32
DeepSeek V4 Pro	**0.12**	0.42	0.47	0.39	0.28	0.16	0.32
Grok 4.6	0.18	0.45	0.37	0.37	0.28	0.14	0.32
GPT-6 Sol	0.18	0.45	**0.35**	0.40	0.27	0.17	0.32
Inkling	0.17	0.43	0.44	0.37	0.27	0.16	0.33
Kimi K3	0.15	0.45	0.46	**0.34**	0.26	0.20	0.33
Claude Fable 5.1	0.17	0.48	0.42	0.39	0.28	0.17	0.33
Beam 1.0	0.20	0.39	0.45	0.38	0.28	0.14	0.33
DeepSeek V4 Flash	0.15	0.47	0.46	0.37	0.26	**0.13**	0.33
GLM-5.3	0.16	0.45	0.44	0.40	0.28	0.14	0.33
Claude Fable 5	0.20	0.45	0.43	0.43	0.28	0.17	0.34
GPT-6 Luna	0.19	0.53	**0.35**	0.38	0.27	0.21	0.34
Qwen3.5 4B (local)	0.15	0.51	0.45	0.39	**0.22**	0.26	0.35

Results

Table 10 reports the pass rate of each built agent. The vanilla pi agents pass 40.0% and 36.0% of the 75 tasks, a mean of 38.0%. The custom-harness agents pass 44.0% and 48.0%, a mean of 46.0%. The custom agents are also cheaper and faster at evaluation time: 0.57 and 0.43 cents of inner-model spend per task against 0.86 and 0.80 for the vanilla agents, and a median of 55 and 50 seconds per task against 107 and 120. Building the agents spent almost none of the $4.00 inner budget (Table 11); no run reached the cap, and all four ended with the researcher declaring itself done. With two seeds per arm the comparison is indicative rather than conclusive: the seed-to-seed spread within an arm (four points) is half the gap between arms, and more seeds would be needed to establish that gap. The harness, the brief and the four agents are released for reproduction.[4]

[4]https://github.com/ruhzi/autoresearch-harness

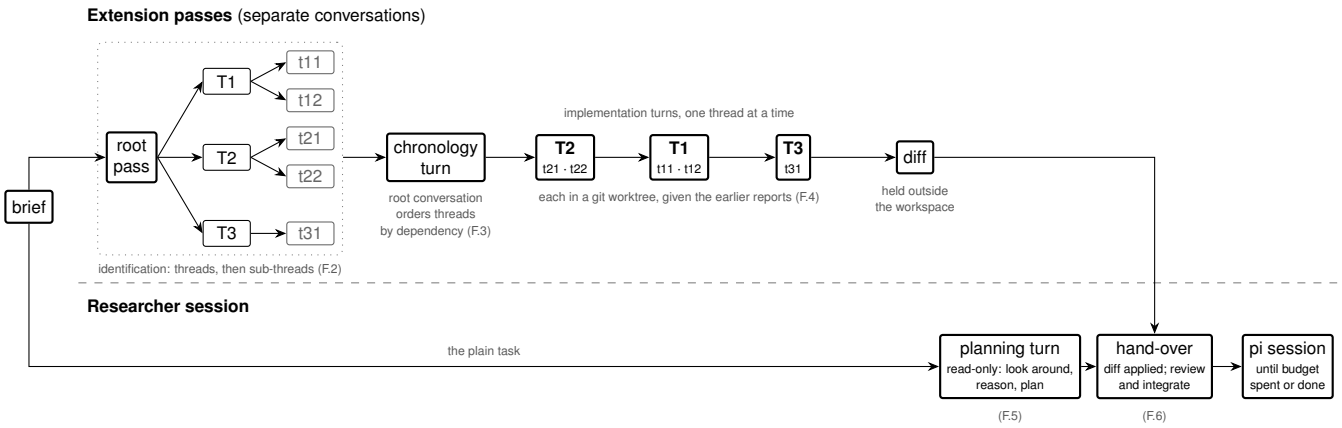


Figure 3: The custom harness. A root pass reads the brief and deduces its critical threads (T), and one further pass per thread deduces its sub-threads (t); depth two, at most six threads per pass. The root conversation then orders the threads by dependency, and each thread's conversation is continued into an implementation turn that builds its sub-threads in that order. The combined diff is held outside the workspace while the researcher plans with edits refused, and is applied when the researcher replies without tool calls; the researcher then reviews and integrates the build, and the run continues as an ordinary pi session. Prompts are in Appendix F.

Table 10: Agents built by each autoresearcher (Custom is pi with the cognizance layer), scored on 75 hard DABstep tasks. Pass rate (%) per seed of the outer run (S1, S2) with the mean over seeds; then, for the same agents, the mean inner-model cost of one evaluation task in US cents and the median wall-clock seconds per task.

	Pass rate (%)			¢ per task		Median s	
Autoresearcher	S1	S2	Mean	S1	S2	S1	S2
Vanilla pi	40.0	36.0	38.0	0.86	0.80	107	120
Custom	**44.0**	**48.0**	**46.0**	**0.57**	**0.43**	**55**	**50**

Table 11: Inner-model spend (USD) of each autoresearcher run while building its agent, against a budget of $4.00 per run.

Autoresearcher	S1	S2
Vanilla pi	0.04	0.02
Custom	0.05	0.03

Limitations

The evaluation set has 42 samples, and the gaps between models in Table 4 are small; more preprints, which the pipeline can draw automatically, would make the ranking firmer.

The failure modes were identified from a close reading of every trace rather than through controlled annotation, and some traces are provider summaries. We consider the reading a faithful account of what the traces do, and controlled annotation of the same traces is the natural next step to confirm the counts.

The answer keys are the numbers the preprints report; ForesightBench takes correct reportage as given, by design.

Three models (Grok 4.6, DeepSeek V4 Pro 0813, GLM-5.3) publish no training cutoff, so their contamination control rests on our assumption that their training data excludes papers submitted one to six days before their release.

The autoresearch-harness comparison is small: two harnesses, two seeds each, one brief, and one benchmark. At this scale we view it as a demonstration and not a definitive evaluation. Larger autoresearch harnesses, more seeds, and more demanding evaluations are left to future work.

Since the retrieval pipeline is configurable and open-source, future work can expand this style of benchmarking into other scientific domains.

References

Allen-Zhu, Z.; and Li, Y. 2024. Physics of Language Models: Part 3.3, Knowledge Capacity Scaling Laws. *arXiv preprint arXiv:2404.05405*.

Bang, Y.; Fielding, K.; Oliver, B.; Birke, B.; Seedat, N.; and Bean, A. M. 2026. ContractScrub: A benchmark for final review of legal contracts. *arXiv preprint arXiv:2608.20204*.

Calderon, N.; Ben-David, E.; Gekhman, Z.; Ofek, E.; and Yona, G. 2026. Empty Shelves or Lost Keys? Recall Is the Bottleneck for Parametric Factuality. *arXiv preprint arXiv:2602.14080*.

Cao, H.; Jaber, M.; Yuan, S.; and Sayed, A. 2026a. Putting Registers to Work: Task Registers for Token Pruning in Vision Transformers. *arXiv preprint arXiv:2608.10989*.

Cao, J.; Li, Q.; Zhang, Z.; Wu, X.; Liu, L.; Li, X.; and Ni, J. 2026b. MarkNull: Model-Agnostic Watermark Removal in AI-Generated Images via On-Manifold Latent Manipulation. *arXiv preprint arXiv:2608.10166*.

Cao, Y.; Wang, G.; Hou, M.; Bu, X.; Liu, D.; and Liu, Y. 2026c. EgoHieraLoc: A Cortically Inspired Hierarchi-

cal Segmentation-Guided Framework for Egocentric Visual Query Localization. *arXiv preprint arXiv:2608.09656*.

Chan, J. S.; Chowdhury, N.; Jaffe, O.; Aung, J.; Sherburn, D.; Mays, E.; Starace, G.; Liu, K.; Maksin, L.; Patwardhan, T.; Weng, L.; and Mądry, A. 2025. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *International Conference on Learning Representations (ICLR)*.

Chen, B.; Zi, H.; Ou, R.; Liu, Y.; Chen, M.; Qi, Z.; Wang, R.; and Liu, G. 2026a. Pragmatic Attack Surface: Vulnerabilities of Implicit Context in Large Language Models. *arXiv preprint arXiv:2608.09551*.

Chen, G.; Li, Z.; Li, R.; Lu, Y.; Shi, H.; Xu, B.; and Huang, J. 2026b. ForestBench: A Unified Graph Framework for Evaluating Multi-Agent Collaboration. *arXiv preprint arXiv:2608.08605*.

Chen, Y.; Benton, J.; Radhakrishnan, A.; Uesato, J.; Denison, C.; Schulman, J.; Somani, A.; Hase, P.; Wagner, M.; Roger, F.; Mikulik, V.; Bowman, S. R.; Leike, J.; Kaplan, J.; and Perez, E. 2025. Reasoning Models Don't Always Say What They Think. *arXiv preprint arXiv:2505.05410*.

D'Alessandro, A.; Mahdavi-Amiri, A.; and Hamarneh, G. 2026. RECOUNT: Reference-guided Counting with Synthetic Visual Exemplars. *arXiv preprint arXiv:2608.20621*.

Desai, R.; Hu, J.; Cabezas, J.; Harsola, N.; Shukla, P.; Ben Chaim, R.; El Assadi, A.; Kamath, O. M.; Faldu, F.; Hebbar, P.; et al. 2026. SWE-Marathon: Can Agents Autonomously Complete Ultra-Long-Horizon Software Work? *arXiv preprint arXiv:2606.07682*.

Ding, T.; Nannapaneni, A.; Liu, B.; and Zhang, L. 2026. Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap. *arXiv preprint arXiv:2608.05179*.

Egg, A.; Iglesias Goyanes, M.; Kingma, F.; Mora, A.; von Werra, L.; and Wolf, T. 2025. DABstep: Data Agent Benchmark for Multi-step Reasoning. *arXiv preprint arXiv:2506.23719*.

Elhagrasy, Y. A.; Hill, I.; and Ivanov, A. 2026. Adaptive Sequential Test Planning for Multi-Mechanism Reliability Qualification via Bayesian Monte Carlo Tree Search. *arXiv preprint arXiv:2608.09622*.

Erramilli, V. 2026. Mechanistic Tomography: Designed Measurement for Control-Oriented Interpretability. *arXiv preprint arXiv:2608.19338*.

Fedus, W.; Zoph, B.; and Shazeer, N. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120): 1–39.

Feng, Z.; Chen, Z.; Ma, J.; Po, Y. T.; Chersoni, E.; and Li, B. 2026. Good Arguments Against the People Pleasers: How Reasoning Mitigates (Yet Masks) LLM Sycophancy. *arXiv preprint arXiv:2603.16643*.

Foster, T. S.; Al Omari, B.; Fu, T.; Mann, T.; Domond, C.; Cipolina-Kun, L.; Gauri, B.; Aghamelu, M.; Goldie, A. D.; Helenowski, E.; Gagnon-Audet, J.-C.; Pepe, A.; Nazir, S.; Izcovich, D.; Levi, N.; Hazra, R.; Hambardzumyan, K.; Baldwin, N.; Li, X.; Josifoski, M.; Giampouras, P.; Jalili Sabet, M.; Sims, A.; Momand, H.; Shavrina, T.; Magka, D.; Weston, J.; Wang, Y.; Goyal, A.; Henriques, J.; Bachrach, Y.; McMilin, E.; and Foerster, J. N. 2026. AI Research Preference Models. *arXiv preprint arXiv:2608.13940*.

Gema, A. P.; Rajani, N.; Saxena, R.; Kwan, W.-C.; and Minervini, P. 2026. Chain-of-Thought Faithfulness of Reasoning Models Varies with Where and How Preference Cues Are Delivered. *arXiv preprint arXiv:2608.29464*.

Godey, N.; and Artzi, Y. 2026. Matryoshka Language Model Suites. *arXiv preprint arXiv:2608.09703*.

Grashei, C.; Gülhan, F.; Legnar, M.; Stögbauer, F.; Weis, C.-A.; Mogler, C.; and Schüffler, P. 2026. An end-to-end-trained vision-language model for native-language prostate pathology report generation. *arXiv preprint arXiv:2608.23143*.

Holzwarth, L.; González-Márquez, R.; and Kobak, D. 2026. Most biomedical publications show signs of LLM-assisted writing. *arXiv preprint arXiv:2608.10715*.

Hua, Y.; Na, H.; Zhou, Y.; Kalose, A.; Ayubcha, C.; and Lian, L. 2026. StagedWorkspace: A Versioned Workspace for Knowledge-Work Agents. *arXiv preprint arXiv:2608.18050*.

Ichihara, Y.; Iwase, N.; Quamar, M. A.; and Komiyama, J. 2026. Privileged Solutions or Context-Induced Teacher Behavior? Dissecting On-Policy Self-Distillation. *arXiv preprint arXiv:2608.09228*.

Jia, J.; Benton, J.; and Easley, E. 2026. Faithfulness as Information Flow: Evaluating and Training Faithful Chain-of-Thought Reasoning. *arXiv preprint arXiv:2605.24286*.

Jiang, X.; Xie, R.; and Tang, M. 2026. SkillSmith: Enhancing Locally Deployed Agents via Automatic Skill Construction and Evolution. *arXiv preprint arXiv:2608.08037*.

Jin, S.; Kim, K.; Lee, J.; and Lee, Y. 2026a. Retrieval-Corrected Conformal Prediction for Time Series. *arXiv preprint arXiv:2608.10553*.

Jin, T.; Dong, S.; Li, H.; and Chou, C.-H. 2026b. Iterative Erasure Count Is Not an Affine-Invariant Concept Dimension. *arXiv preprint arXiv:2608.10566*.

Jo, S.; Ko, D.; Kang, J.; Kwak, Y.; Kwak, J.; and Choi, S. 2026. Flow Straight to Reality: Perceptually Consistent Flow Matching for Efficient Image Restoration. *arXiv preprint arXiv:2608.10544*.

Johnson, A.; Ghizawi, O.; and Minai, A. A. 2026. The Role of Grid Cells in Reducing Spatial Aliasing in Hippocampal Place Representations. *arXiv preprint arXiv:2608.18569*.

Kampe, J. N.; Lee, C. J.; Shen, X.; Lehtiö, A.; von Brandenburg, S.; Nokelainen, O.; Dunson, D. B.; and Ovaskainen, O. 2026. Physics-Informed Learning for Robust Acoustic Localization with Calibrated Uncertainty. *arXiv preprint arXiv:2608.08911*.

Karpathy, A. 2026. autoresearch: Autonomous LLM Research on a Single GPU. https://github.com/karpathy/autoresearch. GitHub repository, March 2026.

Kimi Team. 2026. Kimi K3: Open Frontier Intelligence. *arXiv preprint arXiv:2607.24653*.

Kong, L.; Sun, X.; Chow, W.; Li, L.; Lin, K. Q.; Zhang, X. B.; Wang, S.; Li, R.; Wu, Q.; Gao, W.; et al. 2026. AI

for Auto-Research: Roadmap & User Guide. *arXiv preprint arXiv:2605.18661*.

Kwa, T.; West, B.; Becker, J.; Deng, A.; Garcia, K.; Hasin, M.; Jawhar, S.; Kinniment, M.; Rush, N.; Von Arx, S.; et al. 2025. Measuring AI Ability to Complete Long Software Tasks. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*.

Le, L. D.; Chen, T.; Sadiq, S.; Yin, H.; Jin, M.; and Yu, J. 2026. Towards Efficient Reasoning in LLM-Based Recommender Systems via Model Merging. *arXiv preprint arXiv:2608.10447*.

Levit, M.; Ledgard, J.; Dong, H.; Suryanarayanan, V.; Kolman, E.; Tan, S.; Gan, Q.; and Chowdhary, V. 2026. Privacy-Preserving Data Drift Detection and Recovery for Large-Scale LLM Applications via Proxy Representations. *arXiv preprint arXiv:2608.08245*.

Li, Z. 2026. Beyond Routing: Decoupling Expert Dispatch and Aggregation in Sparse Mixture-of-Experts. *arXiv preprint arXiv:2608.08853*.

Liao, B.; Yu, C.; Yang, Y.; Wang, C.; Song, Z.; Liu, P.; and Wu, T. 2026. Path Integral Value Matching for Linear Quadratic Stochastic Optimal Control. *arXiv preprint arXiv:2608.10777*.

Lin, Y.; Ang, E.; Zhu, E.; Ding, B.; and Zhou, J. 2026. Context as an Environment: Programmatic Context Management for Long-Horizon Agents. *arXiv preprint arXiv:2608.21690*.

Liu, C.; Thiamwong, L.; Fu, Y.; and Xie, R. 2026a. Adaptive Multi-Agent Feature Selection for Personalized Fall Risk Prevention. *arXiv preprint arXiv:2608.18450*.

Liu, X.; Fang, Z.; He, H.; Hong, Y.; and Liu, D. 2026b. Triple Expert Learning from Noisy Labels for Semi-Supervised Vision Foundation Model Adaptation. *arXiv preprint arXiv:2608.09052*.

Liu, Z.; Huang, L.; Su, Z.; Pan, Z.; Han, Z.; Chen, X.; Song, Y.; and Zhang, W. 2026c. SkillLens: Visual Skill Cards for Retrieval-Augmented GUI Action Prediction and On-Policy Distillation. *arXiv preprint arXiv:2608.10775*.

Liu, Z.; Tian, Z.; Wang, X.; Wen, Z.; Xiong, Y.; Lai, Z.; and Li, D. 2026d. Efficient RLVR Scheduling via Graph-Structured Online Difficulty Estimation. *arXiv preprint arXiv:2608.17941*.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.

Luo, X.; Rechardt, A.; Sun, G.; Nejad, K. K.; Yáñez, F.; Yilmaz, B.; Lee, K.; Cohen, A. O.; Borghesani, V.; Pashkov, A.; et al. 2025. Large Language Models Surpass Human Experts in Predicting Neuroscience Results. *Nature Human Behaviour*, 9: 305–315.

Meng, Z.; Meng, Z.; Liu, S.; Xu, H.; Yuan, K.; and Ounis, I. 2026. SSE-Bio: A Structured Self-Evolving Agent with Agentic Retrieval Policy for Multi-Hop Biomedical Reasoning. *arXiv preprint arXiv:2608.22132*.

Mule, S. P.; Garikaparthi, A.; and Patwardhan, M. 2026. Teaching Language Models to Forecast Research Success Through Comparative Idea Evaluation. *arXiv preprint arXiv:2605.21491*.

Nogueira, R.; Bonás, G. K.; Almeida, T. S.; Roque, A.; Pires, R.; Abonizio, H.; Laitz, T.; Larcher, C.; Malaquias Junior, R.; and Piau, M. 2026. Measuring Opinion Bias and Sycophancy via LLM-based Persuasion. *arXiv preprint arXiv:2604.21564*.

Novikov, A.; Vũ, N.; Eisenberger, M.; Dupont, E.; Huang, P.-S.; Wagner, A. Z.; Shirobokov, S.; Kozlovskii, B.; Ruiz, F. J. R.; Mehrabian, A.; et al. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131*.

Ozcam, B.; Onen, I.; Amasyali, M. F.; and Tatli, E. I. 2026. Generating Attacks for LLMs with GFlowNets. *arXiv preprint arXiv:2608.10171*.

Park, J.; Mendes, E.; Stanovsky, G.; and Ritter, A. 2025. Anticipatory Evaluation of Language Models: Predicting LLM Benchmark Scores from Descriptions. *arXiv preprint arXiv:2509.20645*.

Ravuru, C.; and Srivastava, S. 2026. UNMASK: Discovering and Causally Verifying Spurious Shortcuts in Text Classifiers. *arXiv preprint arXiv:2608.09209*.

Saha, D.; Mannan, S. M. A.; Rashid, M. R.; Naswan, R.; and Tahmid, A. 2026. Zero-Shot Traffic Accident Detection via a Coarse-to-Fine VLM-Tracking Pipeline. *arXiv preprint arXiv:2608.08867*.

Sajja, B.; Kleiman-Weiner, M.; Zimmermann, R.; and Zhi-Xuan, T. 2026. Evaluating Rational Contracting in Natural Language. *arXiv preprint arXiv:2608.10475*.

Sehwag, U. M.; Lau, E.; Oskouie, H. E.; Shabihi, S.; Liang, E.; Toledo, A.; Mangialardi, G.; Fonrouge, S.; Cardona, E.-Y. H.; Vergara, P.; et al. 2026. SciPredict: Can LLMs Predict the Outcomes of Scientific Experiments in Natural Sciences? *arXiv preprint arXiv:2604.10718*.

Sharma, M.; Tong, M.; Korbak, T.; Duvenaud, D.; Askell, A.; Bowman, S. R.; Cheng, N.; Durmus, E.; Hatfield-Dodds, Z.; Johnston, S. R.; Kravec, S.; Maxwell, T.; McCandlish, S.; Ndousse, K.; Rausch, O.; Schiefer, N.; Yan, D.; Zhang, M.; and Perez, E. 2024. Towards Understanding Sycophancy in Language Models. In *International Conference on Learning Representations (ICLR)*.

Speck, Z.; and Shepard, A. 2026. Learned, Then Lost: A Measured Single-Example Counterfactual in Pre-training. *arXiv preprint arXiv:2608.19168*.

Sui, H.; Qin, S.; Liao, J.; Jia, X.; Chen, B.; and Liu, Y. 2026. ElasticBack: Stealthy Conditional Backdoor in LLM-Agent Skills via Coupled Trigger-Rule Optimization. *arXiv preprint arXiv:2608.09577*.

Tian, Q.; Yin, H.; Su, H.; Chao, J.; Gu, X.; Kong, Y.; Xia, Y.; and Liu, Z. 2026. ForeSci: Evaluating LLM Agents for Forward-Looking AI Research Judgment. *arXiv preprint arXiv:2606.00644*.

Wang, C.; Wang, L.; Pan, J.; Wang, W.; Ren, W.; Liu, J.; and Cao, X. 2026a. Ultra-High-Definition Restoration Transformers with Correlation Matching Transformation. *arXiv preprint arXiv:2608.20263*.

Wang, H.; Zhu, Y.; Wang, T.; Li, Y.; Dong, J.; Zhang, X.; and Ren, L. 2026b. Physics-informed Diffusion Generative Model for Time-Series Data Synthesis in Dynamic Systems. *arXiv preprint arXiv:2608.10941*.

Wang, J.; Wang, Z.; Guo, Y.; Yang, Y.; and Liang, Y. 2026c. TRACE-Memory: Public-Conditioned Retrieval and Utility-Aware Evidence Admission for Personalized Generation. *arXiv preprint arXiv:2608.08446*.

Wen, J.; Si, C.; Chen, Y.-H.; He, H.; and Feng, S. 2025. Predicting Empirical AI Research Outcomes with Language Models. *arXiv preprint arXiv:2506.00794*.

Weng, Y.; Zhu, M.; Xie, Q.; Sun, Q.; Lin, Z.; Liu, S.; and Zhang, Y. 2025. DeepScientist: Advancing Frontier-Pushing Scientific Findings Progressively. *arXiv preprint arXiv:2509.26603*.

Wijk, H.; Lin, T.; Becker, J.; Jawhar, S.; Parikh, N.; Broadley, T.; Chan, L.; Chen, M.; Clymer, J.; Dhyani, J.; et al. 2024. RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents Against Human Experts. *arXiv preprint arXiv:2411.15114*.

Wu, S.; Lu, P.; Chen, Y.; Bragg, J.; Yamada, Y.; Clark, P.; Clifton, D.; Torr, P.; Zou, J.; and Yu, J. 2026. Scientific Reasoning Does Not Reliably Translate into Scientific Forecasting in Frontier AI. *arXiv preprint arXiv:2605.22681*.

Yen, Y.-F. 2026. Decomposition-Induced Context-Memory Conflict: When Fact-Checking Pipelines Contradict Their Own Source Text. *arXiv preprint arXiv:2608.10627*.

Zamora, C.; Zuniga, H.; Orozco-Rosas, U.; and Picos, K. 2026. Retrieval-Augmented Vision Foundation Models for Robust Leukemia Cell Classification across Multiple Microscopy Datasets. *arXiv preprint arXiv:2608.10657*.

Zechner, M. 2026. pi: a minimal coding agent. https://github.com/badlogic/pi-mono. Npm package `@earendil-works/pi-coding-agent`, version 1.1.0.

Zhang, A. L.; Kraska, T.; and Khattab, O. 2025. Recursive Language Models. *arXiv preprint arXiv:2512.24601*.

Zheng, J.; Zhang, J.; Luo, Y.; Mao, Y.; Gao, Y.; Du, L.; Chen, H.; and Zhang, N. 2026. Can We Predict Before Executing Machine Learning Agents? *arXiv preprint arXiv:2601.05930*.

Zúñiga, N. V. 2026. What Iterated Self-Feeding Probes of Language Models Measure, and a test that separates the construction from the model. *arXiv preprint arXiv:2608.10986*.

Appendix A: Retrieval Pipeline

We draw arXiv e-print LaTeX sources from a fixed submission window and parse them into sections and tables. Five LLM stages then run on each paper. A cheap gate reads only the title, abstract, section outline, and table captions and answers three yes/no questions: is this AI research, does it introduce something of its own, and are results reported. An enumerator lists every controlled comparison, meaning two variants that differ in one technique, as a short description with its benchmarks and metrics but no numbers. The paper is kept outside the model's context window, as in recursive language models (Zhang, Kraska, and Khattab 2025): a root model can grep it, read a page, or send named sections to a subagent, receives one-line digests of what the subagents found, and may not finish until every section and table is covered or skipped with a stated reason. A retrieval agent (OpenRouter slug `z-ai/glm-5.2`) then finds each comparison again and submits start and end anchors instead of text; Python copies the exact source bytes between them for the explanatory passages and the result rows, and both the baseline and the method cell must appear literally in a copied row, so nothing in the payload is model-written and "not found" is an accepted answer. A composer that never sees the method values rewrites the copied passages into a first-person researcher message ending with the baseline numbers; a mechanical check rejects any number, LaTeX command, or leaked method value that is not in the sources, and a judge model, also blind to the method values, then checks the message for hints of the outcome and for fidelity to the passages. Python appends the question, which asks for the value of each (benchmark, metric) pair after switching from the baseline arm to the method arm, and the graded target is that measurement grid. These checks guarantee verbatim text and numbers, not that the two cells form the right comparison, so every sample was manually audited against its paper afterwards.

Appendix B: LLM-Stage System Prompts

The system prompts below are copied verbatim from the pipeline source at the time of writing. Per-paper and per-sample material (title, abstract, outline, spans, baseline values) is supplied in the user turn and is not shown. All calls go through OpenRouter with web search disabled. Model slugs, verbatim from the run records and the pipeline configuration: stage 4 `~deepseek/deepseek-v4-flash-latest`; stage 5 root and enumerators `anthropic/claude-sonnet-5`; stage 6 `z-ai/glm-5.2` (all 1,044 retrieval runs); stage 7 and stage 8 `anthropic/claude-sonnet-5`.

Stage 4: eligibility gate

```
You screen papers for a dataset about AI
research techniques. You are given a
paper's title, arXiv categories, abstract,
section outline, and table captions.
Answer three questions.

1. is_ai_research

   Is this AI research? Judge the paper's
contribution, not its venue -- a paper
   in an astronomy or medical venue counts
if what it contributes is a model or a
   training technique. Not AI research:
classical algorithm design and complexity
   theory, pure mathematics, hardware or
systems work where nothing is learned.

2. introduces_something
```

```
   Does the paper introduce or meaningfully
investigate something of its own
   that can be evaluated through a
controlled comparison -- such as a method,
   module, pipeline, framework, architecture
, training scheme, policy,
   evaluation protocol, component,
modification, or experimental condition?
   This includes novel contributions as well
 as meaningful investigations or
   modifications of existing techniques,
provided they can be compared against
   a baseline or alternative. Not: a
literature survey or position paper that
   only summarises other people's work.

3. has_results

   Are results attached, in some form? The
authors' own numbers, in a table, a
   figure, or the text. Not: only numbers
cited from other papers, and not purely
   qualitative examples with nothing
measured.

Then `reason`: one or two sentences. If any
answer is false, say which and why.
```

Stage 5: root agent

Rendered with the configured page size (2000 characters) and batch limit (60,000 characters).

```
You orchestrate the enumeration of a machine
-learning paper you cannot read
directly. The goal: every CONTROLLED
COMPARISON in the paper ends up in the
environment's findings list. One sample =
one (technique, setting, metric)
triple. You do not judge or filter samples
yourself -- enumerator subagents
do that. Your deliverable is coverage:
submit() succeeds only when every
section and table has been enumerated or
explicitly skipped.

THE ENVIRONMENT

The paper is stored outside your context as
sections s0..sN and tables
t0..tM, in document order. The outline you
are given lists every id with its
kind, title, and size in chars.

YOUR TOOLS

- peek(target, offset): one ~2000-char page
of raw text. For
  orientation only -- reading the paper
yourself is not your job.
- grep(query): literal search across the
whole paper (case-insensitive,
  whitespace-tolerant). Use it to find where
 a table is discussed.
- spawn_enumerator(targets): dispatch a
subagent that reads ONLY those
  sections/tables and appends every
comparison it finds to the findings
  list. You get back a digest (counts and
one line per sample), never the
  full findings. A batch may carry at most
60000 chars total
  (sizes are in the outline).
- coverage(): what is covered, skipped, and
still uncovered.
- findings_brief(): one compressed line per
finding so far.
- skip(targets, reason): mark targets that
genuinely contain no comparisons
  worth enumerating (a bibliography, a code
listing, an acknowledgements
  section). Not a shortcut -- every skip is
recorded in the output.
- submit(): finish. Refused while anything
is neither covered nor skipped.

PLAYBOOK

1. Read the outline. Group related targets
into batches: keep each table in
   the same batch as the section that
discusses it (grep the table's label
   or caption words when unsure). Pack
batches close to the 60000-char
   budget -- fat batches, few calls.
2. Dispatch spawn_enumerator over every
batch. You may issue SEVERAL
   spawn_enumerator calls in one turn; they
run in parallel.
3. Results and appendix sections are where
ablation studies live. They are
   never to be left uncovered, and skipping
them is almost always wrong.
4. Check findings_brief() and coverage(). A
results section that yielded
   zero samples is suspicious -- re-dispatch
 it in a different batch or peek
   to see why. Duplicated findings across
batches are fine (dedupe is
   automatic downstream).
5. If an enumerator fails (its output was
truncated, or an error comes
   back), its targets stay uncovered: split
that batch into smaller ones
   and re-dispatch. Dense, table-heavy
targets yield many samples, so give
   them small batches.
6. submit().

RULES

- No metric values or verbatim quotes
anywhere -- a later stage extracts
  numbers; findings carry only comparisons
and descriptions.
- Do not reconstruct the paper via repeated
peeks. Peek pages are capped and
  your context is small; enumeration belongs
 to subagents.
```

```
- Respond only with tool calls.
```

Stage 5: enumerator subagent

```
You inventory the CONTROLLED COMPARISONS a
machine-learning paper contains.

You are reasoning over the whole paper to
work out how many samples it can
yield and what each one is about in general
terms.

Describe each comparison generally, but
specifically enough that a later stage
handed your description alone could find the
 exact rows in the paper. Two things
are the later stage's job and must NOT
appear here: metric VALUES of any kind,
and verbatim quotes. Everything else that
helps identify a comparison -- what the
technique does, how the paper labels the two
 variants, which table it sits in --
belongs in your description.

A controlled comparison is a pair of
variants differing by ONE identifiable
technique, so the difference is attributable
 to that technique. Each
(technique, methodological setting) pair is
ONE sample. WHERE the comparison
is evaluated -- the benchmarks/datasets AND
the metrics -- are properties of
the sample, NOT what makes it distinct: list
 them all in `benchmarks` and
`metrics`, but never emit separate samples
per benchmark or per metric.

`setting` is the recipe context only (base
model, scale, training config).
Evaluation datasets belong in `benchmarks`.
If two rows differ only in what
they are evaluated on, they are the SAME
sample.

=== HOW MANY SAMPLES ===

- An ablation table with 4 removable
components, scored on 3 benchmarks and
  2 metrics, yields 4 samples -- one per
component, each listing all 3
  benchmarks and both metrics.
- The same recipe evaluated on 3 datasets
yields ONE sample with 3
  benchmarks -- not 3 samples.
- A sweep over a design choice yields one
sample per non-default value.
- Two genuinely different training recipes (
e.g. the technique applied at
  7B vs 70B, or trained under a different
budget) ARE different settings,
  hence different samples.

The technique may be one the paper PROPOSES
or one it merely INVESTIGATES --
an ablated component, a baseline the authors
 ran themselves, a varied
experimental condition (retrieval on/off,
cache policy, decoding strategy).
Both count.

Do not include rows copied from other papers
 with no shared setup, dataset
statistics, or curves where no technique is
being compared.

=== DESCRIBING EACH SAMPLE ===

For each comparison also write a `
description`: one or two sentences saying
what
the technique does and what it is being
compared against.

This is the handle a later stage uses to
find the comparison again, so it has to
be general but discriminating. "A training
strategy is varied" is useless -- a
paper may vary five things. "The heat-kernel
 endpoint filter strength sigma_0 is
raised from 0 (unfiltered endpoint, i.e.
standard flow matching) to 40" points at
exactly one pair of rows.

Name the two variants as the paper labels
them when you can see the labels, e.g.
"full model vs w/o gating", "Shared linear
vs Sample-level". Do not quote long
passages and do not include any numbers --
that is the later stage's job.

=== ORIENTATION: always describe ENABLING
the technique ===

Describe each comparison as adding /
enabling / turning ON the named technique
-- the fuller arm relative to the reduced
arm -- never as removing it. When the
paper labels the two variants "w/o X" and "
full" (or "X off" / "X on",
"baseline" / "+X"), the reference is the
reduced arm and the technique is what
gets ADDED, even though the paper writes the
 ablation as a removal.

  technique "syntactic constraints", arms "w
/o constraints" vs "full":
    the technique is adding the constraints;
 "w/o constraints" is the reference.

=== DO NOT JUDGE THE OUTCOME ===

Do not say whether a comparison helped or
hurt, and do not rank the arms. You
have not seen the numbers, and whether a
move is good depends on the paper's
own goal for each metric -- higher attack-
success is a win in an attack paper
and a loss in a defense paper. That call is
made later, from the extracted
```

```
values. Your job is to inventory WHICH
comparisons exist and WHAT they measure.

=== NO NUMBERS ===

No metric values, deltas, or percentages
anywhere in your output.
Digits inside names are fine ("ResNet-50",
"8 layers"); digits that are
measurements are not.

=== COVERAGE ===

Inventory every comparison across all tables
 and settings, appendix included.
List a comparison even when you cannot tell
how it turned out -- how it turned
out is not your call.
```

Stage 6: retrieval agent

```
You are given ONE comparison that has
already been identified in a
machine-learning paper: a technique, the
setting it runs in, and the metrics
it is reported on. Locate it in the paper
and submit pointers to it. You are
not searching for comparisons -- exactly one
 is described; find that one.

THE PAPER

The paper is stored outside your context as
sections s0..sN and tables
t0..tM (the outline lists every id with kind
, title, and size). Explore with:

- grep(query): literal search, case-
insensitive, whitespace-tolerant.
  The fastest way in: grep the metric name,
the technique, or a row label.
- read(target, offset): one page of raw
LaTeX from a section or table.

POINTERS, NOT TEXT

You never write paper content. You bracket
it: a pointer is
{target, start_anchor, end_anchor}, where
the anchors are SHORT strings
(roughly 3-8 words) copied EXACTLY from what
 grep/read showed you. Each
anchor must occur exactly once in that
target. The system slices the text
between your anchors straight from the
source -- so anchor precision is all
that matters, and any anchor that does not
match comes back to you as an
error to fix.

WHAT TO SUBMIT: submit_found

- spans: pointers to the passages a reader
needs to understand this
  technique and the conditions it was
measured under. Look for, and bracket
  when the paper provides it: the motivation
; the mechanism or formulation;
  the experimental conditions for THIS
comparison (backbone, dataset,
  epochs, protocol -- often stated once at
the top of a section); what the
  baseline variant actually is. Quote what
exists, skip what does not --
  two real spans beat four stretched ones.
Bracket only passages relevant
  to THIS sample's technique.
- evidence: an ARRAY of pointers, each
bracketing a table row pair holding
  values (bracket from the first row's label
 to the end of the second row
  when adjacent; both rows inside the slice)
. Rows usually carry all the
  metrics as columns; different benchmarks
often sit in different rows or
  different tables -- add one evidence
pointer per row pair you read from.
- measurements: one entry PER (BENCHMARK,
METRIC) pair this comparison
  reports. For each: the benchmark name, the
 metric name, and
  baseline_cell / method_cell -- the two
values EXACTLY as they appear in
  the rows (e.g. "71.2"). Each cell must
occur inside one of the evidence
  slices and contain exactly one number.
baseline = the variant WITHOUT
  the technique, method = WITH it. For an
ablation table where the
  technique is X: baseline is "w/o X",
method is "full model". Include
  every pair you can extract; a value
reported only as a range or delta
  is skipped, not guessed.
- baseline_label / method_label: how the
paper labels those two rows.
- per measurement: the benchmark, the metric
 named as the paper names it,
  the unit, and the two cells. Do not judge
whether a move is good.
- isolates_single_technique: false if the
two rows differ in more than one
  respect. Still submit, but flag it.
- confidence: low if you had to choose
between plausible row pairings.

WHEN TO GIVE UP: submit_not_found

Returning not_found with a reason is a
correct and useful answer -- the
described comparison is not in this paper,
or the values exist only as a
range, a plus/minus, or a relative delta (
not extractable), or the
description is too vague to match a specific
 pair of rows. Do not force a
match: a wrong number becomes a training
label nothing downstream can catch.
```

```
Do not judge whether the change is good or
bad; that is computed
arithmetically downstream. Respond only with
 tool calls.
```

Stage 7: composer

```
You rewrite a researcher's source material
into a single first-person
message: someone neutrally describing an
idea they have been brainstorming
and the exact experiment they have set up,
ending with their current
baseline result. A question is appended
later; you write only the body.

The message must read as the researcher's
own thinking, in a neutral tone:
an opener like the one you are given, then "
my setup is...", "my baseline
gets...". Never present it as coming from a
paper, and never name authors
or venues.

HARD RULES

1. Preserve every technical detail from the
source passages: the mechanism,
   the formulation, every hyperparameter,
dataset, model, resolution, epoch
   count. Losing a detail is as bad as
inventing one.
2. Copy every number EXACTLY as it appears.
Never introduce a number that
   is not in the source material.
3. Copy every equation and LaTeX expression
EXACTLY as written.
4. State the baseline results (every metric'
s label and value you are
   given) as the current starting point.
5. NO outcome language: do not say or hint
that the idea works, fails, or
   changes anything. You do not know the
outcome. Stating the goal or
   motivation behind the design is fine;
stating results is not.
6. Do not add expectations, hedges about
likely results, or requests -- the
   question is appended separately.
```

Stage 8: verification judge

```
You audit a training sample for a benchmark.
 The sample is a researcher
describing an idea and an experimental setup
, ending with two questions
about what will happen. You are given the
researcher's message and the
original source passages it was written from
. Answer two things.

1. outcome_reference

   Does the message anywhere state, imply,
or hint at the RESULT of the
   experiment -- that the idea works, fails,
 improves things, or how the
   numbers came out? The baseline result is
allowed (it is the given
   starting point). The questions at the end
 are allowed. Statements of
   motivation or intent ("the goal is to
reduce memory") are allowed.
   What is NOT allowed is any signal about
what actually happened when the
   idea was tried. Be strict: a hedge like "
which should help convergence"
   or "encouragingly" counts as a reference.

2. faithful

   Compare the message against the source
passages. Every number
   (hyperparameters, dataset sizes, the
baseline value), every equation,
   and every setup detail in the message
must match the source exactly.
   Report each mismatch in fidelity_issues:
dropped details, changed
   values, reworded formulas. Extra framing
language is fine; changed
   facts are not.

Judge only what is in front of you. Do not
guess the experiment's outcome.
```

Appendix C: A Complete Sample

Sample `2608.09209-s051` (Ravuru and Srivastava 2026) from the evaluation set, reproduced verbatim from the corpus. The prompt is sent verbatim as the user turn, after a fixed system turn, the same for every sample and every model, that asks for one point estimate per slot in a closing JSON block. The target is the graded measurement grid, which the model never sees. This sample passed every automated check and was not hand-edited.

System turn

````
You are forecasting the outcome of a machine
-learning experiment that has already been
run, from the researcher's description of it
 alone. The results are not in front of you
and you cannot look them up; estimate them.

Think it through in as much depth as you
want first. Then close your reply with a
single fenced JSON block, with nothing after
 it:

```json
{"predictions": [{"slot": "<label, copied
exactly>", "value": <number>}]}
```

Rules for that block:
````

```
1. One entry per metric the researcher asks
you to predict, in the order they are asked,
 and no extra entries.
2. "slot" is the metric's label copied
exactly as it appears in the question,
including any parenthesised unit.
3. "value" is a bare JSON number on that
metric's own scale -- the same scale the
researcher quoted their baseline on. No
percent signs, no units, no thousands
separators, no ranges, no strings, no nulls.
4. Commit to a single best point estimate
for every metric. Predicting no change is a
legitimate answer; refusing to answer is not
.

The prose before the block is yours to use
however you like. Only the block is read
mechanically.
```

Prompt (user turn)

```
I've been spending cycles on isolating how
much of the gain in our BERT DFR debiasing
pipeline actually comes from the discovery
stage versus the downstream recipe. The idea
 is to hold the debiasing recipe fixed and
only vary how candidate features/groups are
discovered, comparing a few discovery
baselines: PMI (top-10 single-token
statistics), PMI+SFV (PMI candidates passed
through our statistical validator), and LLM-
only (prompt-only LLM candidates with no
validation funnel). The point is to check
whether cheaper feature mining could produce
 the same groups as the full pipeline, which
 would mean the discovery stage isn't
earning the gains on its own. My setup is
BERT-base-uncased with DFR on CivilComments-
WILDS, run across 3 seeds, comparing the
full UNMASK discovery pipeline against the
LLM-only variant (no statistical/causal
validation) under this fixed downstream
recipe. My baseline gets, for CivilComments-
WILDS with the LLM-only discovery variant,
WGA = 65.1% and avg-acc = 89.71%.

My question about this setup: after
switching from LLM-only (my baseline above)
to UNMASK, predict the resulting value for
each: CivilComments-WILDS WGA (in %);
CivilComments-WILDS avg-acc (in %).
```

Target


```
{
  "measurements": [
    {
      "benchmark": "CivilComments-WILDS",
      "metric": "WGA",
      "baseline_value": 65.1,
      "method_value": 71.84,
      "unit": "%"
    },
    {
      "benchmark": "CivilComments-WILDS",
      "metric": "avg-acc",
      "baseline_value": 89.71,
      "method_value": 90.99,
      "unit": "%"
    }
  ]
}
```


Appendix D: Research-Area Taxonomy

Each preprint carries one research-area tag, assigned by hand from its title and abstract and applied to every sample built from it. Table 12 gives the six areas together with the rule used to assign them; where a preprint straddles two areas the rule decides, so an attack on image watermarks (Cao et al. 2026b) is tagged safety rather than vision, model merging for an LLM recommender (Le et al. 2026) is tagged applied ML rather than LLM training, and a benchmark whose subject is a multi-agent system (Chen et al. 2026b) is tagged agents rather than evaluation. The six areas fall into two groups. For agents, safety, and evaluation studies the forecast concerns the behaviour of a language model or an LLM-based system; for vision, applied, and LLM-training work it concerns a trained model's score on a conventional benchmark. Table 13 lists and cites every corpus preprint with its tag and the number of corpus samples it contributes.

Appendix E: Additional Results

Table 14 repeats Table 9 with the median per-sample error in place of the mean. Table 15 extends Table 8 to all three failure modes.

Appendix F: Autoresearch-Harness Brief and Prompts

The brief and the prompts below are copied verbatim from the arena and the harness source at the time of writing. The brief is the task both arms receive; nothing in it names a benchmark item. The prompts are those of the custom arm's extension (the Autoresearch-Harness section); the vanilla arm runs pi with its stock prompts only. Fields that the extension fills at run time (the brief, thread ids, earlier reports) are shown as angle-bracketed placeholders, and the thread cap is rendered with the configured value (6).

F.1 The brief

The file `BRIEF.md`, given to the researcher as its first prompt. `$INNER_API_BASE`, `$INNER_MODEL` and `$INNER_API_KEY` are environment variables set in the researcher container.

```
# Brief: build a long-horizon agent

You are a researcher. Your job is to build
an agent that can take a question
about a body of files, work through those
files with a shell
```

Table 12: The six research areas and their definitions.

Area	Definition and assignment rule
Vision & multimodal	Vision or vision-language models on vision tasks: classification, segmentation, restoration, counting, detection, token pruning in ViTs, medical imaging, report generation. Rule: the input modality is images or video and the metric is a vision benchmark score.
Safety, attacks, interpretability & auditing	Studies that probe, attack, audit or measure a model's behaviour or internals rather than improve a task score: jailbreaks, backdoors, watermark removal, attack generation, probes and their transfer, concept erasure, shortcut discovery, mechanistic measurement. Rule: the proposed method is adversarial, or the measured quantity is what the model internally does.
LLM agents, memory & context	An LLM agent is the system under study: skill libraries, memory and retrieval policies, context management, GUI or knowledge-work agents, multi-agent orchestration, LLM agents in games. Rule: the thing being measured is an agent loop, even when the paper is framed as a benchmark.
Applied & scientific ML	A learned model applied to a non-LLM scientific or engineering problem: acoustic localisation, stochastic control, time-series conformal prediction, reliability test planning, computational neuroscience, clinical risk prediction, recommender systems. Rule: the metric is a domain quantity (metres, control error, interval width), not an LLM behaviour.
LLM training & architecture	Pre-training, post-training and architecture of language models themselves: nested model suites, self-distillation, RL scheduling, pre-training counterfactuals, MoE routing. Rule: the intervention changes how a language model is trained or built, and the metric is a standard LM score (perplexity, accuracy, cross-entropy).
LLM evaluation studies	How LLMs are evaluated or how their use is measured, with no agent loop involved: benchmark construction, judge calibration and drift, usage-rate measurement, cost and latency of reasoning modes. Rule: the paper is about the evaluation apparatus or a population measurement, not a model or method.

```
and Python over many steps, and commit to
one answer. The agent lives in
‘agent/‘. The agent is to be used for long
horizon reasoning tasks.

## What the agent is for

Long-horizon work over a workspace of files:
 investigations that take tens of
tool calls, where the agent has to find out
what is in the workspace, read the
documentation that explains it, follow a
chain of evidence across several
files, compute things along the way, check
its own intermediate results, and
arrive at a conclusion it can defend. The
workspaces are collections of data
files and their documentation; the questions
 are quantitative and the
documentation decides what the terms mean.
The agent is given the workspace
and the question when it is run, and nothing
 before that. Some questions have
no valid answer in the workspace, and
recognising that is part of the job.

Expect every task to be well beyond what a
single model turn can do. The tasks
are the hard kind: each needs many steps,
and none is a simple lookup.

## Conditions the agent runs under

- One fresh process per task, with no memory
 of previous tasks.
- Shell and Python 3.12, with pandas and
numpy available and nothing else
  installable. No network.
- A language model it can call at ‘
$INNER_API_BASE‘ with the name
  ‘$INNER_MODEL‘, through an OpenAI-
compatible chat API with tool calling. This
  is the only model it has.

## Your budget

You have 4.00 US dollars of model calls for
the whole job: the calls your agent
makes to ‘$INNER_API_BASE‘ whenever you run
it, whether on a practice task you
built, on a quick check, or on anything else
. A single run of the agent on a
task of this genre costs on the order of a
cent or two. When the budget is
spent, the endpoint returns HTTP 402 and the
 job is over; whatever is in
‘agent/‘ at that moment is your deliverable.
You can read the remaining budget at any
time:

    curl -s -H "authorization: Bearer
$INNER_API_KEY" http://gateway:8080/inner/
usage

We evaluate the deliverable afterwards on
```

Table 13: Area assignment of all 42 corpus preprints, with the number of corpus samples each contributes.

Area	arXiv	Title	Samples
Vision & multimodal	2608.08867	Zero-Shot Traffic Accident Detection via a Coarse-to-Fine VLM-Tracking Pipeline (Saha et al. 2026)	2
	2608.09052	Triple Expert Learning from Noisy Labels for Semi-Supervised Vision Foundation Model Adaptation (Liu et al. 2026b)	8
	2608.09656	EgoHieraLoc: A Cortically Inspired Hierarchical Segmentation-Guided Framework for Egocentric Visual Query Localization (Cao et al. 2026c)	22
	2608.10544	Flow Straight to Reality: Perceptually Consistent Flow Matching for Efficient Image Restoration (Jo et al. 2026)	10
	2608.10657	Retrieval-Augmented Vision Foundation Models for Robust Leukemia Cell Classification across Multiple Microscopy Datasets (Zamora et al. 2026)	3
	2608.10989	Putting Registers to Work: Task Registers for Token Pruning in Vision Transformers (Cao et al. 2026a)	24
	2608.20263	Ultra-High-Definition Restoration Transformers with Correlation Matching Transformation (Wang et al. 2026a)	22
	2608.20621	RECOUNT: Reference-guided Counting with Synthetic Visual Exemplars (D'Alessandro, Mahdavi-Amiri, and Hamarneh 2026)	9
	2608.23143	An end-to-end-trained vision-language model for native-language prostate pathology report generation (Grashei et al. 2026)	1
Safety, attacks, interpretability & auditing	2608.09209	UNMASK: Discovering and Causally Verifying Spurious Shortcuts in Text Classifiers (Ravuru and Srivastava 2026)	25
	2608.09551	Pragmatic Attack Surface: Vulnerabilities of Implicit Context in Large Language Models (Chen et al. 2026a)	9
	2608.09577	ElasticBack: Stealthy Conditional Backdoor in LLM-Agent Skills via Coupled Trigger-Rule Optimization (Sui et al. 2026)	2
	2608.10166	MarkNull: Model-Agnostic Watermark Removal in AI-Generated Images via On-Manifold Latent Manipulation (Cao et al. 2026b)	4
	2608.10171	Generating Attacks for LLMs with GFlowNets (Ozcam et al. 2026)	1
	2608.10566	Iterative Erasure Count Is Not an Affine-Invariant Concept Dimension (Jin et al. 2026b)	8
	2608.10627	Decomposition-Induced Context-Memory Conflict: When Fact-Checking Pipelines Contradict Their Own Source Text (Yen 2026)	3
	2608.10986	What Iterated Self-Feeding Probes of Language Models Measure, and a test that separates the construction from the model (Zúñiga 2026)	4
	2608.19338	Mechanistic Tomography: Designed Measurement for Control-Oriented Interpretability (Erramilli 2026)	5
LLM agents, memory & context	2608.08037	SkillSmith: Enhancing Locally Deployed Agents via Automatic Skill Construction and Evolution (Jiang, Xie, and Tang 2026)	8
	2608.08446	TRACE-Memory: Public-Conditioned Retrieval and Utility-Aware Evidence Admission for Personalized Generation (Wang et al. 2026c)	6
	2608.08605	ForestBench: A Unified Graph Framework for Evaluating Multi-Agent Collaboration (Chen et al. 2026b)	2
	2608.10475	Evaluating Rational Contracting in Natural Language (Sajja et al. 2026)	1
	2608.10775	SkillLens: Visual Skill Cards for Retrieval-Augmented GUI Action Prediction and On-Policy Distillation (Liu et al. 2026c)	3
	2608.18050	StagedWorkspace: A Versioned Workspace for Knowledge-Work Agents (Hua et al. 2026)	4
	2608.21690	Context as an Environment: Programmatic Context Management for Long-Horizon Agents (Lin et al. 2026)	1
	2608.22132	SSE-Bio: A Structured Self-Evolving Agent with Agentic Retrieval Policy for Multi-Hop Biomedical Reasoning (Meng et al. 2026)	2
Applied & scientific ML	2608.08911	Physics-Informed Learning for Robust Acoustic Localization with Calibrated Uncertainty (Kampe et al. 2026)	3
	2608.09622	Adaptive Sequential Test Planning for Multi-Mechanism Reliability Qualification via Bayesian Monte Carlo Tree Search (Elhagrasy, Hill, and Ivanov 2026)	10
	2608.10447	Towards Efficient Reasoning in LLM-Based Recommender Systems via Model Merging (Le et al. 2026)	15
	2608.10553	Retrieval-Corrected Conformal Prediction for Time Series (Jin et al. 2026a)	13
	2608.10777	Path Integral Value Matching for Linear Quadratic Stochastic Optimal Control (Liao et al. 2026)	6
	2608.10941	Physics-informed Diffusion Generative Model for Time-Series Data Synthesis in Dynamic Systems (Wang et al. 2026b)	1
	2608.18450	Adaptive Multi-Agent Feature Selection for Personalized Fall Risk Prevention (Liu et al. 2026a)	16
	2608.18569	The Role of Grid Cells in Reducing Spatial Aliasing in Hippocampal Place Representations (Johnson, Ghizawi, and Minai 2026)	1
LLM training & architecture	2608.08853	Beyond Routing: Decoupling Expert Dispatch and Aggregation in Sparse Mixture-of-Experts (Li 2026)	7
	2608.09228	Privileged Solutions or Context-Induced Teacher Behavior? Dissecting On-Policy Self-Distillation (Ichihara et al. 2026)	29
	2608.09703	Matryoshka Language Model Suites (Godey and Artzi 2026)	6
	2608.17941	Efficient RLVR Scheduling via Graph-Structured Online Difficulty Estimation (Liu et al. 2026d)	11
	2608.19168	Learned, Then Lost: A Measured Single-Example Counterfactual in Pre-training (Speck and Shepard 2026)	3
LLM evaluation studies	2608.08245	Privacy-Preserving Data Drift Detection and Recovery for Large-Scale LLM Applications via Proxy Representations (Levit et al. 2026)	3
	2608.10715	Most biomedical publications show signs of LLM-assisted writing (Holzwarth, González-Márquez, and Kobak 2026)	1
	2608.20204	ContractScrub: A benchmark for final review of legal contracts (Bang et al. 2026)	2

Table 14: Median error by research area.

Model Samples	Vision 9	Safety 9	Agents 8	Applied 8	LLM-train 5	LLM-eval 3	All 42
GPT-6 Astra	0.04	0.21	**0.26**	0.26	**0.16**	0.13	**0.17**
DeepSeek V4 Pro	0.05	0.33	0.36	0.27	0.17	0.13	**0.17**
GPT-5.6 Sol	**0.03**	0.29	0.30	0.25	**0.16**	0.17	0.18
Kimi K3	0.07	0.51	0.39	0.29	0.17	0.14	0.18
DeepSeek V4 Flash	0.09	0.41	0.32	0.37	**0.16**	0.17	0.18
Claude Fable 5.1	0.04	0.45	0.39	0.27	0.17	0.16	0.19
GPT-6 Luna	0.04	0.49	0.28	0.25	**0.16**	0.21	0.19
GPT-6 Sol	0.05	0.43	0.31	**0.21**	0.19	0.14	0.20
Grok 4.6	0.06	0.46	0.32	0.34	**0.16**	**0.08**	0.21
Inkling	0.06	0.34	0.32	0.22	**0.16**	0.17	0.21
Beam 1.0	0.06	0.28	0.32	0.28	0.17	0.19	0.21
Claude Opus 5.5	0.07	**0.20**	0.30	0.28	0.17	0.14	0.22
Qwen3.5 4B (local)	0.09	0.51	0.31	0.29	**0.16**	0.25	0.22
GLM-5.3	0.04	0.41	0.36	0.29	**0.16**	0.15	0.23
Claude Fable 5	0.10	0.24	0.37	0.33	0.17	0.16	0.25

Table 15: All failure modes by model.

Model	Traces	Mode 1			Mode 2			Mode 3			None
		n	with	without	n	with	without	n	with	without	
GPT-6 Luna	42	4	0.11	0.36	6	0.55	0.30	**0**	–	0.34	**32**
GPT-6 Astra	42	7	0.33	0.30	4	0.61	0.27	3	0.22	0.31	30
Claude Opus 5.5	41	**2**	0.19	0.31	**3**	0.60	0.28	8	0.30	0.31	29
GPT-6 Sol	42	5	0.21	0.34	**3**	0.68	0.29	5	0.17	0.34	29
Beam 1.0	42	8	0.50	0.29	7	0.58	0.28	8	0.60	0.27	26
GPT-5.6 Sol	42	10	0.29	0.32	7	0.55	0.27	4	0.61	0.28	23
GLM-5.3	42	6	0.18	0.36	9	0.65	0.25	17	0.39	0.29	19
DeepSeek V4 Flash	42	9	0.33	0.33	10	0.53	0.27	15	0.39	0.30	18
Claude Fable 5.1	39	**2**	0.54	0.32	6	0.61	0.28	14	0.24	0.38	17
Inkling	42	17	0.39	0.28	10	0.70	0.21	20	0.43	0.24	15
Claude Fable 5	38	**2**	0.19	0.34	4	0.52	0.31	20	0.33	0.34	15
DeepSeek V4 Pro	41	10	0.53	0.25	6	0.49	0.29	28	0.32	0.32	10
Qwen3.5 4B (local)	42	10	0.52	0.29	27	0.45	0.17	9	0.34	0.35	10
Grok 4.6	42	34	0.32	0.32	9	0.58	0.25	6	0.38	0.31	7
Kimi K3	42	11	0.26	0.35	9	0.61	0.25	35	0.31	0.40	5
All	621	137	0.35	0.32	120	0.56	0.27	192	0.35	0.31	285
Share		22%			19%			31%			46%

```
our own budget.

## How you work

State lives in this repository: `results.tsv
` for your log, `practice/` for
anything you build to test with, `agent/`
for the deliverable. Your context may
be reset while you work, so keep the log
current enough to resume from. Write
`DONE` in the repository root when you judge
 that no further experiment is
worth its cost.

## What you start from

`agent/` contains a minimal working agent.
It is a starting point, not a
recommendation.

## Deliverable

The directory `agent/`, honouring the
command-line contract in
`agent/README.md` exactly: the flags, the
answer-file format, exit code 0. Run
the contract check described there before
you finish. An agent that fails it
scores zero on every task.
```

F.2 Identification passes

System prompt appended to every identification pass (root and sub-thread), run with read-only tools.

```
You identify the load-bearing threads of a
task.

A thread is one distinct thing the task
requires to be worked on: a requirement, a
constraint, a condition the work runs under,
 a resource, or a risk. A thread is
load-bearing if getting it wrong, or
ignoring it, changes whether the task
succeeds.

Rules:
- Work only from the task and the files it
points to. Read them fully before answering.
  Do not rely on what tasks like this
usually involve.
- Anchor every thread in the text. Quote the
 exact words it comes from, and name the
file.
  If a thread is implied rather than stated,
 quote the words it is implied by.
- Each thread must be distinct. Do not list
the same thing twice in different words.
- Return at most 6 threads. If there are
more, keep the most load-bearing.
- If the part you are given is already a
single concrete thing that can be worked on
  directly, return no threads.
- You only identify. Do not plan, solve, or
suggest how to do anything.

End your answer with exactly one JSON block
in this shape and nothing after it:
{"threads": [{"title": "<short name>",
              "quote": "<exact words from
the source>",
              "source": "<file the quote is
from>",
              "why_load_bearing": "<what
goes wrong in the outcome if this is missed
>"}]}
```

User message of the root pass.

```
Here is a task, exactly as it was given:

<<<
<the brief, verbatim>
>>>

Identify the load-bearing threads of this
task.
```

User message of a sub-thread pass (one per first-level thread).

```
Here is a task, exactly as it was given:

<<<
<the brief, verbatim>
>>>

Earlier passes broke this task into threads.
 You are given one of them:

Path: <thread id path, e.g. T2 > T2.1>
<each thread on the path: id, title, quote,
source file, why it bears load>

Identify the load-bearing threads within <
thread id> only: the distinct things that
must be
worked on to get this thread right. Stay
inside this thread; other threads are
handled
separately.
```

F.3 Chronology turn

Delivered as one more turn of the root conversation.

```
The threads you identified will now be
worked on one at a time, each by a different
 person,
each building on the work of the ones before
. Put them in the order they should be
worked on:
what has to exist before something else can
be built on it comes first. Order by
dependency,
not by importance.

<one line per thread: id, title, why it
bears load>

End your answer with exactly one JSON block
```

```
in this shape and nothing after it:
{"order": ["<thread id>", "<thread id>",
...]}
Every id exactly once.
```

F.4 Implementation turns

Delivered as one more turn of each first-level thread's conversation, in canon order, with full tools in a git worktree.

```
Now implement the threads you identified
within <thread id and title>, in this
workspace:
<one line per sub-thread: id, title>

The threads of this task are being
implemented one at a time, each by a
different person, in
this order:
<all threads in canon order, marked done /
you / after you>

The work of the threads before yours is
already in this workspace. Their reports:

<the reports of the implementers before this
 one, or "(none: you are first)">

What is there is earlier work on other
threads of the same task, each done by
someone who saw
that one thread; none of it is final. Your
own reading of the brief governs your thread
. You have
the same authority over earlier threads'
work as over your own: when your thread
needs it
different, change it, including a design
choice an earlier thread presented as
settled. Read a
file before you edit it. Keeping earlier
work is also a decision: for every earlier
decision your
thread touches or depends on, check it
against the brief yourself and say whether
you kept it or
changed it. Every change to earlier work
must be named in your report: the file, the
thread the
work came from, and why your thread needs it
 different.

When you are done, finish with a report. For
 each of your threads, report:
- what you changed (files and what the
change does),
- why, in terms of that thread,
- its state: done, partly done, or not
started,
- what is left and anything the reviewer
should check.
Then list every earlier decision you checked
 against the brief, each marked kept or
changed with
the thread and the reason; if you changed
nothing of earlier work, say so and say why
it held.
```

Steer delivered once, after the first successful edit at or above the configured share of the context window (60%); the implementer is not told of this rule in advance.

```
Stop implementing now. Review what you have
changed so far against the sub-threads you
were working on. For each sub-thread, report
:
- what you changed (files and what the
change does),
- why, in terms of that sub-thread,
- its state: done, partly done, or not
started,
- what is left and anything the reviewer
should check.
End with the report; do not make further
changes.
```

F.5 Researcher planning turn

Appended to the researcher's first prompt. Edits and writes are refused until it replies.

```
Before you do anything else: read the brief
and look around this repository as much as
you need,
but change nothing yet. Then reason out loud
, in your reply, about what you would build,
 how you
would test it, and what could go wrong. Once
 you have replied, you will be told more and
 can
start the work.
```

F.6 Hand-over

Delivered to the researcher after it replies to the planning turn without tool calls, once the implementers' diff is applied.

```
You are not starting from scratch. While you
 were thinking, the result of earlier work
was
applied to this repository: before you began
, the task was broken into load-bearing
threads
(a tree: each thread names what it bears and
 why, with sub-threads under it), and <N>
implementers worked on those threads one
after another, each building on the ones
before. Their
changes are now in the working tree (all of
them: .harness/implement.diff). You are the
final
reviewer and integrator: the deliverable is
still yours, and your job now is to turn
their work
into one coherent, working result that does
what the brief asks.
```

```
What you wrote in your last reply was
written without seeing any of this.
Reconcile it in two
passes, and write both down.

Pass 1, your reply against the thread tree.
For each thing you said you would build or
test, name
the thread that covers it, or say that no
thread does. Then the other way: for each
thread, say
whether you had it, and if not, whether the
thread is right that it bears load. A thread
 you did
not think of may be a real gap in your
thinking, or may be weight on something that
 does not
matter; decide which.

Pass 2, your reply against the code as built
. Treat what you wrote as a design, not a
note: go
through it point by point. For each point,
establish whether the build does it, does it
differently, or does not do it. Where it
does not, you build it. Where it does it
differently,
decide which is right and say why.

Disagreement is expected, and you decide it.
 The implementers each saw one thread of the
 task, in
sequence, and none of them saw the whole.
You do. Nothing they built is settled: any
decision in
the tree or in the code, including one every
 implementer after it inherited, is yours to
 overturn
when you judge it should be, and you need no
 permission to change their work. Where
their choices
conflict with what you would have done, your
 judgment governs: change the work, and
record what
you changed and why. Treat their reports as
claims to verify, not as results.

Approval is not a result. The implementers'
own tests passing shows their work does what
 they
meant; it does not show the deliverable does
 what the brief asks. You have the budget,
and it is
there to be spent: build tasks of your own
that look like the ones the brief describes,
 run the
deliverable on them, judge it the way it
will be judged, and change what fails.
Ending this job
without having changed anything is a finding
 you have to justify, not a default. After
the
review, carry on with the job as the brief
describes, within the budget that remains.

Threads (full detail: .harness/threads.md)
<the thread tree index>

Implementers, in the order they worked (full
 reports: .harness/implement.md)
<one line per implementer, in canon order:
thread, edit count, files>
```

Appendix G: One Sample, Every Model

The sample. Sample `2608.08446-s005` is drawn from TRACE-Memory (arXiv 2608.08446; Wang et al. 2026c), a personalised-generation paper. It is one sample of 42, shown as a qualitative illustration; its mean error across the fifteen models (0.94) is the fourth highest in the evaluation set. The prompt describes a small policy for memory querying and evidence selection, initialised by SFT and then optimised, and an LLM+LLM baseline in which the frozen response generator also makes the memory decisions. It asks for TRACE-Memory's signed preference advantage over that baseline in six cells: Controlled (Goodreads) and Natural (Goodreads, Amazon Reviews, Reddit) tasks, each with DeepSeek-V4-Flash, Gemini 3.1 Flash-Lite and GPT-5.4 as the generator. The baseline is 0.0 in every cell by construction, and the prompt does not state the scale. The reported advantage is negative in five cells (-0.02 to -0.12) and positive only on Natural tasks with GPT-5.4 ($+0.06$).

Where every model failed. No model scored below 0.86 on the sample (Table 16).

- *Direction.* All fourteen frontier models predicted a positive advantage in all six cells, so each had the sign right only on Natural/GPT-5.4. Qwen3.5 4B predicted positive or zero values on the Controlled cells and negative values on the Natural cells; it had the sign right on Natural/DeepSeek-V4-Flash and Natural/Gemini and wrong on the other four, including the one positive cell.
- *Task ordering.* Thirteen of the fourteen frontier models (all but DeepSeek V4 Pro) predicted a larger advantage on Controlled than on Natural tasks for every generator. The reported advantage is lower on Controlled tasks for Gemini and GPT-5.4.
- *Generator ordering.* The same twelve models placed their smallest predicted advantage on Natural/GPT-5.4, the only cell in which the policy wins.
- *Scale.* Five models (Claude Opus 5.5, Claude Fable 5.1, GPT-5.6 Sol, DeepSeek V4 Pro and DeepSeek V4 Flash) answered on a percentage-point scale, with values from 0.4 to 19.5. The other nine answered between -0.4 and $+0.30$.

The Modes column of Table 16 lists each trace's tags from the main analysis; the passages below, each quoted verbatim, also include traces not tagged with the mode in question.

Mode 1: unfaithful chain of thought. DeepSeek V4 Pro reasons "suggests maybe trained policy is not necessarily better", "Thus some negatives likely." and "Let's include a negative for GPT-5.4 controlled", then "I'd rather avoid negative because policy is optimized" and "All positive." Grok 4.6's

Table 16: Every model's prediction on sample `2608.08446-s005` (TRACE-Memory; Wang et al. 2026c): signed preference advantage of the trained memory policy over the LLM+LLM baseline, reported on a $[-1, 1]$ scale, for Controlled (Goodreads) and Natural (Goodreads, Amazon Reviews, Reddit) tasks with DeepSeek-V4-Flash (DS), Gemini 3.1 Flash-Lite (Gem) and GPT-5.4 (GPT) as the frozen generator. Values above 1 are models that answered in percentage points. Sign: cells with the reported sign. Error: capped relative error on the sample. Modes: failure modes tagged in the trace.

	Controlled (Goodreads)			Natural (three sources)					
Model	DS	Gem	GPT	DS	Gem	GPT	Sign	Error	Modes
Reported	-0.02	-0.11	-0.12	-0.06	-0.07	$+0.06$	–	–	–
GPT-6 Astra	0.18	0.20	0.13	0.10	0.12	0.07	1/6	0.86	2
Claude Opus 5.5	15	19	11	8	11	6	1/6	1.00	2
GPT-5.6 Sol	16.8	19.5	10.7	8.4	10.1	5.6	1/6	1.00	2
DeepSeek V4 Pro	1.3	2.6	0.4	3.7	5.3	1.7	1/6	1.00	1, 3
Grok 4.6	0.14	0.18	0.09	0.07	0.11	0.04	1/6	0.89	1, 2
GPT-6 Sol	0.23	0.20	0.16	0.13	0.11	0.08	1/6	0.89	2
Inkling	0.25	0.22	0.18	0.20	0.17	0.13	1/6	1.00	2, 3
Kimi K3	0.24	0.30	0.16	0.13	0.18	0.07	1/6	0.86	2, 3
Claude Fable 5.1	14	18	9	8	11	4	1/6	1.00	2
DeepSeek V4 Flash	4.3	3.6	2.7	3.4	2.8	2.0	1/6	1.00	–
GLM-5.3	0.10	0.12	0.08	0.05	0.06	0.03	1/6	0.92	2, 3
Claude Fable 5	0.18	0.20	0.10	0.10	0.12	0.05	1/6	0.86	2, 3
GPT-6 Luna	0.11	0.09	0.06	0.08	0.06	0.04	1/6	0.89	–
Beam 1.0	0.062	0.048	0.015	0.039	0.027	0.006	1/6	0.98	1, 2, 3
Qwen3.5 4B (local)	0.3	0.1	0.0	-0.3	-0.4	-0.2	2/6	1.00	2

reasoning says "the generator choice doesn't swing the result much"; its answer says "The magnitude of that advantage shrinks as the generator" and gives six values that differ from those listed in its reasoning. GLM-5.3: "a research question, not a proposal claiming wins" and "Need not assume method wins."; Kimi K3: "With GPT strongest, maybe near zero or negative." and "TRACE could hold up but not necessarily beat."; DeepSeek V4 Flash: "TRACE advantage smaller, maybe near zero or negative." followed by "Not negative." All five committed positive values in every cell.

Mode 2: lapse in cognizance of the setup. Priors that a trained policy beats a prompted model: Claude Opus 5.5 "should beat a frontier LLM on the same inputs"; Grok 4.6 "That separation usually gives tighter control and better generalization"; GLM-5.3 "trained smaller models beat prompted LLMs"; DeepSeek V4 Flash "TRACE-Memory should improve because it's optimized for final preference"; DeepSeek V4 Pro "optimized for the specific retriever, so likely better"; Claude Fable 5.1 "a frontier LLM tends to over-retrieve/over-inject evidence". Priors about the task split: Claude Fable 5.1 "where the trained policy has the clearest edge"; Claude Fable 5 "a trained policy should shine more there"; Kimi K3 "On controlled tasks, optimized policy likely clear win."; Inkling "especially on controlled tasks where queries are structured". Priors about the generators: GPT-6 Astra "stronger baseline memory decisions should leave less room"; GPT-6 Luna "the largest gains against weaker memory-decision baselines"; Claude Opus 5.5 "Gemini 3.1 Flash-Lite is the weakest of the three"; Qwen3.5 4B "GPT-5.4: Strongest overall, will likely dominate" and "multimodal strength compensates for single-task specialization". Stated setup elements read differently: GPT-5.6 Sol credits the policy's "constrained evidence budget", where the prompt states that both methods share evidence budgets; GPT-6 Sol cites "performance metrics like DeepSeek-V4-Flash at +19.5%", a figure absent from the prompt. On scale, Claude Opus 5.5 writes "fits a percentage scale better than a fraction" and Claude Fable 5.1 "leaning toward percentage points given the one-decimal formatting".

Mode 3: proposal bias. Claude Fable 5: "Papers proposing trained memory policies almost always report positive". Inkling: "Unlikely, since they propose TRACE-Memory as an improvement." Kimi K3: "TRACE-Memory likely proposed method and likely outperforms baseline.", "'progressively optimized' suggests improvement." and "Signed advantage maybe positive across all due paper claims." GLM-5.3: "In paper methods usually show wins.", "User likely expects positive." and "A proposed method likely optimized, should be positive." DeepSeek V4 Pro: "However, if no change, why ask?", "They likely expect positive because they are proposing TRACE-Memory." and "The user likely would prefer all positive?"